

LAYTH KASHOU

Phone: +33 7 49 82 37 51
45 Bis bd Jourdan, 75014 Paris, France
Email: laythkashou@gmail.com

EDUCATION

Master 2 Optimization, Université Paris Saclay, Orsay, France *2024 - 2025*

- GPA: 15.72/20
- Courses Included: Operation Research, Convex Analysis and Optimization, Game Theory and Applications, Derivative-Free Optimization, Nonsmooth Optimization, Optimal Transport.

Master of Mathematics, An-Najah National University, Nablus, Palestine *2021 - 2024*

- GPA: 3.88/4 (Excellent)
- Courses Include: Operation Research, Mathematical Statistics, Applied Statistics, Algebra, Numerical Analysis, Real Analysis, Complex Analysis.
- Thesis: Radius of Convergence for (Multivariable) Hypergeometric Functions.

Bachelor of Mathematics, An-Najah National University, Nablus, Palestine *2018- 2021*

- GPA: 3.83/4 (Excellent)
- Courses Included: Linear Programming, Computer and Mathematics, Methods of Statistics, Probability Theory, Linear Algebra, Modern Algebra , Modern Analysis, Differential Equations.
- Thesis: Extension Fields.

Elementary and Secondary Education, Haja Secondary School, Palestine *2007 - 2018*

- Scientific Stream
- GPA: 96.3% (Excellent)

HONORS AND FUNDS

- Fully Funded France Excellence Master Level Scholarship *2024 - 2025*
- Fully Funded Erasmus+ scholarship at University Paris-Saclay, France *01/06/2023 - 31/07/2023*
- Fully Funded Erasmus+ scholarship at University of L'Aquila, Italy *05/09/2022 - 11/09/2022*
- Partially Fund for Excellence in Bachelor of Mathematics at An-Najah National University *2018 - 2021*

WORK EXPERIENCE

Ministry of Education, Qalqilia, Palestine *2022 - 2024*

- Teacher of Mathematics for Secondary Stage at Industrial School, Qalqilia, Palestine
- Teacher of Mathematics for Secondary Stage at Amatin Secondary School, Qalqilia, Palestine

INTERNSHIPS

Five months internship under the supervision of Prof. Andre ROSSI, Prof. Evgeny GUREVSKY and Prof. Rui S. SHIBASAKI, Université Paris-Dauphine, Paris, France *10/04/2025 - 10/09/2025*

- Analyzing the stability radius in simple assembly line balancing problems (SALBP) under different norms.
- Using Benders Decomposition to solve the MILP formulations to maximize the stability radius
- Proposing algorithms for efficiently solving the decomposed models.
- Improving skills in computing (Gurob, Python).

Two months internship under the supervision of Prof. Samuel FRIOT, University Paris-Saclay, Orsay, France 01/06/2023 - 31/07/2023

- Deriving new transformation formulas for the Kampé de Fériet function.
- Finding new methods to calculate the radius of convergence for multivariable hypergeometric functions.
- Improving skills in computing (Mathematica, Python).

SUMMER SCHOOLS

Blended Intensive Program (BIP) in M²ISI - Mathematical Models in Social Innovation, University of L'Aquila, L'Aquila, Italy. 05/09/2022 - 11/09/2022

- The program was about applied mathematics viewed as an interdisciplinary area at the interface with engineering, economy, life and social sciences
- It covered several applied topics including models for biology, opinion formation, traffic and pedestrian flows and data science.
- It addressed different methodologies, such as the analysis of partial differential equations, numerical calculus and simulations and kinetic modeling.

PROJECTS

Benchmarking CMA-ES Under Artificial Noise on the Noiseless Test Suite Dec 2024 – Feb 2025

- Completed as part of the *M2 Optimization* program at Université Paris-Saclay, for the *Derivative-Free Optimization* course.
- Evaluated the performance and robustness of the CMA-ES algorithm under artificially injected noise using the BBOB (Black-Box Optimization Benchmarking) noiseless test suite.
- Implemented the experiments on the COCO platform and analyzed results across varying noise levels, functions, and dimensions.
- Demonstrated that the algorithm maintains stable behavior under varying levels of positive noise across all tested functions and dimensions in the COCO platform.

Electric Vehicle Smart Charging for Decarbonization Jan 2025 - Feb 2025

- Completed as part of the *M2 Optimization* program at Université Paris-Saclay, for the *Game Theory and Applications* course.
- Designed decentralized algorithms for smart EV charging aimed at minimizing CO_2 emissions using real-world data from the French electricity grid.
- Applied strategies such as Water-Filling and Best Response Dynamics in a game-theoretic setting to optimize individual and collective charging decisions.

A Hybrid of Mirror Descent and Dual Averaging Algorithms Mar 2025- Apr 2025

- Completed as part of the *M2 Optimization* program at Université Paris-Saclay, for the *Online Learning: Links with Optimization and Games* course.
- Derived regret bounds for a hybrid algorithm combining Mirror Descent and Dual Averaging methods.
- Provided theoretical guarantees for both Lipschitz and smooth convex optimization settings.
- Performed numerical experiments comparing the hybrid algorithm with classical Mirror Descent and Dual Averaging to assess empirical performance.

COURSE WORK

- Cours de français langue étrangère, Institut Français, Ramallah, Palestine. 2024
- Participation in the L^AT_EX course for total 6 training hours in An-Najah National University, Palestine. 2023
- Participation in the *R* course for total 15 training hours in An-Najah National University, Palestine. 2022

VOLUNTEERING EXPERIENCE AND ACTIVITIES

Peers Lecturing Help Session Math

2020 - 2022

- A collaborative Work gathering group of math students who dedicated themselves to help other students from different majors in various math courses.
- Prior Exams Sessions and discussions for the following courses: Calculus, Statistics, and Abstract Algebra.

SKILLS

Computer Skills

- Operating systems: Windows.
- Programming languages and programs: Python, Gurobi, CPLEX, R, Matlab, Mathematica, Excel, Lindo, Minitab.
- Editor: L^AT_EX.

LANGUAGES

- English: Education Language
- Arabic: Native Language

REFERENCES

- **Andre Rossi:** Professor, University Paris Dauphine - PSL
Email: andre.rossi@dauphine.psl.eu
- **Celine Gicquel:** Assistant Professor, University Paris Saclay
Email: celine.gicquel@lri.fr
- **Samuel Friot:** Professor, University Paris Saclay
Email: samuel.friot@ijclab.in2p3.fr
- **khalid Adarbeh:** Assistant Professor, An-Najah National University
Email: khalid.adarbeh@najah.edu

Benders decomposition for robust balancing of production lines

Layth Kashou¹, André Rossi², Evgeny Gurevsky³, Rui S. Shibasaki⁴

¹ IMO, Université Paris-Saclay, Orsay, France

² LAMSADE, Université Paris-Dauphine - PSL, France

³ LS2N, Université de Nantes, France

⁴ MIS, Université de Picardie Jules Verne, Amiens, France

Keywords : *production lines, balancing, uncertainty, stability radius, robustness, MILP, Benders decomposition, Branch-and-Benders-Cut.*

1 Problem

In [2], Rossi et al. study a robust variant of the simple assembly line balancing problem (SALBP), which is a strongly $\mathcal{NP}$ -hard classic combinatorial optimization challenge in the field of production system design. The typical SALBP aims to assign a given set of assembly tasks to a given set of workstations subject to various technical constraints (such as task precedence relations, cycle time, and others), while meeting production efficiency goals. However, in practice, task processing times are uncertain due to human performance variability, micro-stoppages, machine conditions, or product changes. These uncertainties may cause violations of the cycle time constraint and lead to costly line interruptions. Thus, the authors seek to design an assembly line that remain feasible even when task durations deviate from their nominal values.

Traditional stochastic or fuzzy approaches model the variability of task processing time using probability or possibility distributions, but these require information that is often unavailable at the design stage. Instead, Rossi et al. in [2] adopt a robust optimization perspective based on the stability radius concept. This measure quantifies the maximum allowable deviation in task processing times for which a given line configuration remains feasible. In other words, the stability radius evaluates how “robust” a solution is in the face of uncertainty: the larger the radius, the more stable the configuration. In [3, 4], the authors only calculate the stability radius for given line designs (SALBP-1 and SALBP-2). Rossi et al. in [2] go further and define a new optimization problem, called SALBP-S, whose objective is to find not only a feasible line, but one with the maximum possible stability radius. They formalize SALBP-S under two different mathematical norms: the ℓ_1 -norm (sum of deviations) and the ℓ_∞ -norm (maximum deviation). For each case, they prove that the problem is strongly $\mathcal{NP}$ -hard.

2 Solution approach

In this work, we start by studying the MILP formulation, proposed in [2], for maximizing the stability radius for assembly lines with a fixed cycle time and a limited number of workstations, aiming to improve its computational efficiency. We apply Benders decomposition to the original model and verify its correctness by comparing the obtained optimal values with those reported in [2], where the authors use the Xpress Mosel MIP solver. Across all tested instances, our decomposition produced identical objective values, confirming the validity of the approach.

One of the main bottlenecks in Benders decomposition, especially for large instances, is repeatedly solving the dual sub-problem at every iteration if the optimal solution has not been reached. To reduce computational effort, we develop four tailored algorithms for solving the dual sub-problem. We embedded these algorithms in place of the Gurobi MIP solver within the Benders loop. In our experiments, the tailored approaches consistently reduce overall solution time compared with using Gurobi to solve the dual sub-problem.

We also observed that whenever optimality has not yet been reached, the master problem has to be resolved from scratch after each newly added cut, which is also computationally expensive.

To address this, we adopted a Branch-and-Benders-Cut (BBC) algorithm that integrates Benders decomposition directly into the branch-and-bound tree through the framework of **Lazy Constraint** callbacks. Combined with our dual sub-problem algorithms, the BBC approach further decreases the computational effort in our experiments.

We further enhance the efficiency of the BBC algorithm by developing a heuristic algorithm, inspired by the work of [1]. This heuristic iteratively generates improved feasible solutions, from which we derive and pre-inject optimality cuts into the master problem before running the BBC algorithm. The results show that this step alone does not consistently speed up the solution process. Guided by an analysis of these outcomes, we introduce a reformulated master problem that explicitly incorporates information extracted from the proposed heuristic. In our tests, this reformulation, combined with the BBC algorithm and our tailored dual algorithms, delivers the best overall performance.

Based on the MILP formulation of [2], we develop two additional MILP models to maximize the stability radius and apply the same methodology. Each is solved using Benders decomposition, with specialized algorithms for the dual sub-problem. We further strengthen the framework by integrating the BBC algorithm and the heuristic that generates solutions with increasing stability radii. The best heuristic solution is then used to reformulate the master problem, significantly improving overall efficiency.

To compare performance, we test all three formulations, recording the best stability radius achieved within a fixed time limit and the corresponding optimality gap. Both the original (non-decomposed) models and their Benders-decomposed variants with acceleration techniques are evaluated. Overall, the results do not show a consistent efficiency advantage for Benders decomposition with accelerations across the three formulations. Therefore, we conclude that for the three formulations, Benders decomposition does not provide a systematic advantage over the non-decomposed formulations.

Finally, this work suggests a deeper analysis of the Benders optimality cuts to make the decomposition approach more competitive. When applying the BBC algorithm, each time a node with a feasible integer solution is reached, we run the proposed heuristic algorithm in parallel with the BBC process. In this step, the heuristic is executed with the variables fixed according to the branching decisions at that node. Based on the solution obtained by the heuristic, we generate a corresponding Benders optimality cut and add it to all active nodes. This strategy is expected to enhance the efficiency of the overall decomposition approach.

References

- [1] E. Gurevsky, O. Battaia, and A. Dolgui. Stability measure for a generalized assembly line balancing problem. *Discrete Applied Mathematics*, 161(3):377–394, 2013.
- [2] A. Rossi, E. Gurevsky, O. Battaia, and A. Dolgui. Maximizing the robustness for simple assembly lines with fixed cycle time and limited number of workstations. *Discrete Applied Mathematics*, 208:123–136, 2016.
- [3] Y. N. Sotskov, A. Dolgui, and M.-C. Portmann. Stability analysis of an optimal balance for an assembly line with fixed cycle time. *European Journal of Operational Research*, 168(3):783–797, 2006.
- [4] Y. N. Sotskov, A. Dolgui, N. Sotskova, and F. Werner. Stability of optimal line balance with given station set. In A. Dolgui, J. Soldek, and O. Zaikin, editors, *Supply Chain Optimisation: Product/Process Design, Facility Location and Flow Control*, volume 94 of *Applied Optimization*, chapter 10, pages 135–149. Springer US, 2005.

Abstract

In our work, we addressed the problem of maximizing the robustness of simple assembly lines with a fixed cycle time and a limited number of workstations, where robustness is measured by the stability radius. Three different mathematical formulations are used for this problem, each modeled as a Mixed-Integer Linear Program (MILP). For each formulation, we investigate decomposition-based solution approaches.

We start by applying classical Benders decomposition to separate the problem into a master problem and a dual subproblem. We then enhance the solution process by designing tailored algorithms for efficiently solving the dual subproblem, and by integrating these algorithms within the Branch-and-Benders-Cut (BBC) algorithm. Further improvements to the BBC algorithm are achieved by reformulating the master problem. The proposed reformulation integrates information derived from the proposed heuristic algorithm.

Each approach is computationally evaluated on benchmark instances, with performance compared in terms of solution quality and computation time. Finally, we make a comparison between the three formulations, both without decomposition and with decomposition complemented by the proposed techniques, in terms of robustness and computational efficiency.

Paris, le 19 septembre 2025,

Prof. André Rossi
Université Paris Dauphine – PSL
LAMSADE – UMR CNRS 7243
Place du Maréchal de Lattre de Tassigny
75775 Paris Cedex 16
FRANCE
andre.rossi@dauphine.psl.eu

Lettre de soutien du tuteur académique du mémoire

Monsieur Layth Kashou a effectué son stage de fin d'études au LAMSADE (UMR CNRS 7243), à l'Université Paris Dauphine - PSL sur un sujet intitulé "Benders Decomposition for Robust Balancing of Production Lines", sous la supervision de Rui Shibasaki (Université de Picardie Jules Verne à Amiens), Evgeny Gurevsky (Nantes Université) et André Rossi (Université Paris Dauphine - PSL) entre avril et septembre 2025. Etudiant en master d'informatique à l'Université Paris Saclay, il a bénéficié d'un financement de l'Université de Picardie Jules Verne.

Le profil de Layth Kashou est plutôt orienté vers les mathématiques, mais il a choisi un sujet de recherche opérationnelle qui nécessitait une part importante de développement informatique, impliquant notamment l'utilisation d'API pour intervenir sur la résolution mise en œuvre par les solveurs comme CPLEX et Gurobi. Au plan méthodologique, il a également eu à se familiariser rapidement avec des méthodes numériques comme la décomposition de Benders, qui n'étaient pas explicitement au programme de sa formation de master. Il a cependant relevé ces défis avec succès, et a pu contribuer à l'étude d'un problème de ligne d'équilibrage dont on cherche à maximiser un indicateur de robustesse appelé rayon de stabilité. Layth Kashou s'est remarquablement bien familiarisé avec le problème, ses différentes formulations, et avec la décomposition de Benders qu'il a implémenté de diverses manières. Bien sûr, ce travail nécessite d'être poursuivi pour améliorer la qualité des solutions obtenues, mais il a le mérite d'ouvrir des perspectives et d'éclairer les pistes à explorer pour que la décomposition de Benders se révèle compétitive dans ce contexte. Nous apportons notre soutien à sa participation au Prix du Mémoire de Master en Recherche Opérationnelle et Aide à la Décision organisé par la ROADEF.

Evgeny Gurevsky,



André Rossi,



Rui Shibasaki,



UNIVERSITÉ PARIS-SACLAY

LABORATOIRE DE MATHÉMATIQUES D'ORSAY

M2 INTERNSHIP

Benders Decomposition for Robust Balancing of Production LinesF

Author:

Layth KASHOU

Supervisors:

Prof. Andre ROSSI

Dr. Rui SA SHIBASAKI

Dr. Evgeny GUREVSKY

October 29, 2025

Contents

1	Introduction	6
2	Maximizing the Robustness for Simple Assembly Lines with Fixed Cycle Time and Limited Number of Workstations	9
2.1	Mixed-Integer Linear Programming Formulation for Maximize the Stability Radius . . .	9
2.2	Benders Decomposition	14
2.3	Benders Decomposition on MILP Formulation to Maximize the Stability Radius	16
2.3.1	Computational Results	18
2.4	Tailored Algorithms for Solving the Dual subproblem	21
2.4.1	Computational Results	26
2.5	Branch-and-Benders-Cut Algorithm on MILP Formulation to Maximize the Stability Radius	27
2.5.1	Computational Results	27
2.6	Improving the Efficiency of the Branch-and-Benders-Cut Algorithm	29
2.6.1	Branch-and-Benders-Cut Algorithm with Pre-injected Cuts from a Heuristic Algorithm	29
2.6.1.1	Computational Results	30
2.6.2	Branch-and-Benders-Cut Algorithm with the Master Problem Reformulation Based on the Heuristic Algorithm	34
2.6.2.1	Computational Results	35
3	Second Formulation for Maximizing the Robustness of Simple Assembly Lines with Fixed Cycle Time and Limited Number of Workstations	41
3.1	Mixed-Integer Linear Programming Formulation for Maximize the Stability Radius . . .	41
3.2	Benders Decomposition on the Second Formulation to Maximize the Stability Radius .	43
3.2.1	Computational Results	45
3.3	Branch-and-Benders-Cut Algorithm with the Master Problem Reformulation Based on the Heuristic Algorithm on the Second Formulation to Maximize the Stability Radius .	46
3.3.1	Computational Results	47

4	Third Formulation for Maximizing the Robustness of Simple Assembly Lines with Fixed Cycle Time and Limited Number of Workstations	49
4.1	Mixed-Integer Linear Programming Formulation for Maximize the Stability Radius . . .	49
4.2	Benders Decomposition on the Third Formulation to Maximize the Stability Radius . .	50
4.2.1	Computational Results	52
4.3	Branch-and-Benders-Cut Algorithm with the Master Problem Reformulation Based on the Heuristic Algorithm on the Third Formulation to Maximize the Stability Radius . .	53
4.3.1	Computational Results	54
5	Comparison Between the Three Formulations for Maximizing the Stability Radius	55
6	Conclusion	59
A	Algorithm for Solving the Dual Subproblem	61
B	Description of Instances Used in the Experimental Study	65

Dedication

To my loving parents, my dad Yahya Kashou and my mother Tahani Kashou, whose unwavering support and encouragement have been my driving force. Their belief in me has been a constant source of strength, and I am forever grateful for their endless love and sacrifices.

To my siblings, Lara Kashou, Ladin Kashou, Lujayn Kashou and Leen kashou, who have always stood by me, cheering me on through every challenge, your support has meant the world to me.

To my beloved homeland, Palestine, the land of martyrs and unyielding dignity where the blood of the innocent has flowed across its breadth and length. To steadfast Gaza, its courageous people, and its unwavering resistance, you remain the pulse in my heart and the compass that guides me, even as I stand far away in France.

Acknowledgement

Foremost, my sincere thanks to Allah, the Almighty, for granting me countless blessings and for giving me the strength, patience, and determination to accomplish this humble work.

I would like to express my deepest gratitude to my supervisors: Prof. André Rossi, for his invaluable guidance and support throughout this internship; and Dr. Rui Sa Shibasaki and Dr. Evgeny Gurevsky, for their time, advice, and constructive feedback during the course of my research.

I am also deeply grateful to all those who have contributed to my academic journey, from the dedicated teachers of my school days to the esteemed professors at my previous university, An-Najah National University, and to the professors at my current institution, Université Paris-Saclay (Institut de Mathématique d'Orsay). Your dedication and expertise have had a profound impact on my development.

Moreover, my heartfelt thanks go to Université Paris-Dauphine and its laboratory LAMSADE for granting me the opportunity to conduct this internship in a supportive and inspiring research environment.

Last but not least, I extend my gratitude to everyone who has supported me, directly or indirectly, in completing this internship.

Abstract

This report addresses the problem of maximizing the robustness of Simple Assembly Lines with a fixed cycle time and a limited number of workstations, where robustness is measured by the stability radius. Three different mathematical formulations are used for this problem, each modeled as a Mixed-Integer Linear Program (MILP). For each formulation, we investigate decomposition-based solution approaches.

First, we apply classical Benders decomposition to separate the problem into a master problem and a dual subproblem. We then enhance the solution process by designing tailored algorithms for efficiently solving the dual subproblem, and by integrating these algorithms within the Branch-and-Benders-Cut (BBC) algorithm. Further improvements to the BBC algorithm are achieved by reformulating the master problem. The proposed reformulation integrates information derived from the heuristic algorithm.

Each approach is computationally evaluated on benchmark instances, with performance compared in terms of solution quality and computation time. In the last chapter, we provide a comparison of the three formulations—both without decomposition and with decomposition complemented by the proposed techniques—in terms of robustness and computational efficiency.

Chapter 1

Introduction

In the world of industrial manufacturing, few innovations have shaped production as profoundly as the *assembly line*. Introduced at scale by Henry Ford in the early 20th century (see [1]), the assembly line revolutionized production by dividing labor into discrete, repeatable tasks performed in a fixed sequence. This innovation enabled dramatic increases in productivity, efficiency, and consistency, facilitating the rise of mass production across diverse sectors such as automotive, electronics, food processing, and even software development.

At its core, an assembly line is a structured system (see, for example, [2, 3]) in which the units move sequentially through a series of workstations. Each station is responsible for executing a specific set of tasks on every unit that passes through. This production model is grounded in the principle of division of labor, whereby specialized workers or machines perform well-defined subtasks, enabling high throughput and standardization.

A simple assembly line, in particular, represents a fundamental variant of this model. It is a flow-oriented manufacturing system designed to produce large volumes of a single product type. The system comprises a series of linearly ordered workstations connected by a conveyor belt or similar transfer mechanism. Product units are injected at the beginning of the line and transferred synchronously from one station to the next until they exit the line fully assembled. Each workstation repetitively performs its assigned tasks on each unit in the exact same manner.

Several structural assumptions are typically adopted in modeling simple assembly lines (see [3, 4]):

- Only one unit can be processed simultaneously at the workstation and only one workstation at a time can handle the unit;
- Tasks must respect technological precedence constraints, which prevent arbitrary execution orders;
- The tasks of any workstation are performed sequentially one by one without splitting;
- There is neither buffer stock nor parallel workstation and, as a consequence, the transfer of all the units situated on the line is implemented in a synchronized manner, i.e., all units are moved from their current workstations to the next ones simultaneously.

Designing an assembly line is a big deal, it usually requires a lot of investment and careful planning.

One of the most important challenges during this stage is figuring out how to assign the tasks across the workstations, a challenge known as the *balancing problem*. An *Assembly Line Balancing Problem (ALBP)* (see, for example, [2, 3]) is one of assigning the tasks to the stations while optimizing some criterion and not violating a number of possible restrictions or constraints. These constraints include, first, *precedence constraints*, which require that certain tasks be completed before others can begin. This is typically modeled using a *Directed Acyclic Graph (DAG)*, where nodes represent tasks and an arc from task i to task j indicates that task j cannot start before task i finishes. Second, there is the *cycle time constraint*, which ensures that the total processing time of tasks assigned to any workstation does not exceed a given value called *cycle time*. Finally, limitations of the available space for assembling may be naturally translated into restraints on the maximal number of workstations to be installed.

Based on the objective, the *Simple Assembly Line Balancing Problem (SALBP)* is classified into several types (see [2, 3, 4]). In SALBP-1, the goal is to minimize the number of workstations required to complete all tasks, given a fixed cycle time. In contrast, SALBP-2 aims to minimize the cycle time, assuming a fixed number of workstations is already determined. A more flexible variant, SALBP-E (Efficiency), does not fix either the number of workstations or the cycle time in advance. Instead, the objective is to maximize the line efficiency, where it seeks for a line configuration that minimizes the following expression: the number of used workstations multiplied by the working time on the most loaded one.

One of the critical factors that must be considered when designing Simple Assembly Line Balancing Problems (SALBP), is the *task time variability*. In practice, task times are often not precisely known, and only nominal or estimated values are typically used. This uncertainty arises from several practical factors. In manual assembly lines, for example, task execution times depend on the operator's performance, which may vary due to differences in work pace, skill level, fatigue, and motivation. Additionally, product specifications and workstation characteristics may evolve over the life cycle of the line, often driven by changing customer demands or updates in material technology. Finally, various delays and micro-stoppages can occur during task execution.

Any of these events may happen at any time during the operation of the line and can lead to costly interruptions, particularly if the cycle time is exceeded. Therefore, to ensure a robust and reliable line configuration suitable for long-term use, task time variability should be accounted for during the line balancing stage.

To address this, some tasks referred to as *uncertain tasks* are modeled with variable processing times. In this context, researchers have proposed a robustness indicator known as the *stability radius* to evaluate a solution in such a situation, first introduced in the context of SALBP-1 and SALBP-2 in [3, 5, 6]. Given a feasible solution, the stability radius measures the maximum allowable deviation of the uncertain task times from their nominal values for which the feasibility (or optimality) of the solution is still guaranteed. This indicator serves as a meaningful measure of robustness: the larger the stability radius of a given solution, the more robust the line configuration. In [7] the results from SALBP-1 and SALBP-2 was extended to SALBP-E and in another contribution [8], the authors proposed a robust version for a more complex assembly line balancing problem, also considering task time uncertainty.

Finally, in [3], A. Rossi et al. introduced a new variant of the assembly line balancing problem,

referred to as SALBP-S, where S stands for stability radius maximization. This is the type of problem we will focus on in our report. The authors consider an assembly line configuration with a fixed cycle time, a limited number of workstations, and precedence constraints, and aim to find a feasible task assignment that maximizes the stability radius. To evaluate the robustness of a given solution, the authors define the stability radius under two norms: the ℓ_1 -norm and the ℓ_∞ -norm. For each norm, they prove that the corresponding optimization problem (denoted by P_1 and P_∞ respectively) is strongly $\mathcal{NP}$ -hard and propose a Mixed-Integer Linear Programming (MILP) formulation to solve it.

In our report we will consider the optimization problem P_∞ with corresponding to maximizing the stability radius under ℓ_∞ -norm and our aim to solve it efficiently by employing Benders Decomposition along with other complementary techniques. The rest of the report is organized as follows:

- In Chapter 2, we begin by introducing the original Mixed-Integer Linear Programming (MILP) formulation for maximizing the stability introduced in [3]. We then provide a brief introduction to Benders decomposition and apply it to our MILP formulation to solve it more efficiently. To further enhance the efficiency of the Benders decomposition, we propose four algorithms for solving the resulting dual subproblem faster than a solver (Gurobi). We then improve it further by applying the Branch-and-Benders-Cut (BBC) algorithm to our decomposition with the proposed algorithms for the dual subproblem. To increase the efficiency of the BBC algorithm, we use a heuristic algorithm that produces solutions with increasing stability radius to propose an alternative reformulation of the master problem obtained from Benders decomposition. All these developments are supported by computational results, which demonstrate that each technique yields an improvement in efficiency over the previous step.
- In Chapters 3 and 4, we apply the same techniques used in Chapter 2, but now to two other formulations for maximizing the stability radius.
- In Chapter 5, we provide a comparison between the original three formulations for maximizing the stability radius without decomposition and the same three formulations with decomposition, complemented by the techniques mentioned above. This analysis allows us to determine which approach is more effective for solving the problem of maximizing the stability radius.

Chapter 2

Maximizing the Robustness for Simple Assembly Lines with Fixed Cycle Time and Limited Number of Workstations

In this chapter, we present a Mixed-Integer Linear Programming (MILP) formulation for maximizing the stability radius in the SALBP-S, following the model proposed in [3]. We then apply Benders Decomposition to solve this MILP formulation more efficiently. To address the computational challenges of the decomposition, we introduce tailored algorithms for solving the resulting dual subproblem effectively. Building on this, we implement a Branch-and-Benders-Cut (BBC) algorithm that incorporates our proposed dual algorithms. Finally, we enhance the BBC framework by integrating a heuristic algorithm designed to reformulate the master problem in an alternative way, leading to a significant reduction in overall computational time.

2.1 Mixed-Integer Linear Programming Formulation for Maximize the Stability Radius

Before introducing the MILP formulation for maximizing the stability radius of SALBP-S, as described in [3], we first present the following notations, definitions that will facilitate a better understanding of the SALBP-S problem.

- $V = \{1, 2, \dots, n\}$ is the set of necessary assembly tasks;
- $W = \{1, 2, \dots, m\}$ is the set of available workstations;
- t_j is a nominal (or estimated) processing time of the task $j \in V$;
- T is the cycle time which considered to be fixed for all workstations;

- $G = (V, A)$ is a directed acyclic graph representing the precedence constraints, where the tasks correspond to the nodes and A is the set of arcs.

By defining V_k^s to be the set of tasks assigned to the workstation k of the solution s , an assignment of tasks to workstations is feasible if and only if it satisfies the following four constraints:

$$V = \bigcup_{k \in W} V_k^s, \quad (2.1)$$

$$V_p^s \cap V_q^s = \emptyset, \quad \forall p, q \in W, \quad \text{where } p \neq q, \quad (2.2)$$

$$(i, j) \in A \Rightarrow k \leq l, \quad \text{where } i \in V_k^s, j \in V_l^s, \quad (2.3)$$

$$\sum_{j \in V_k^s} t_j \leq T, \quad \forall k \in W. \quad (2.4)$$

where the constraints (2.1) and (2.2) ensures that each task is assigned to exactly one workstation, constraint (2.3) states that if we have an arc $(i, j) \in A$ then the task j cannot start before the task i is completed and constraints (2.4) is referred to be the cycle time constraint, where the total working time of any workstation is not greater than the cycle time T .

As we mentioned in the introduction, some tasks referred to as *uncertain tasks* are modeled with variable processing times. So, A. Rossi et al in [3], distinguish between two types of uncertain tasks:

- A set $\tilde{V}^1 \subseteq V$ of *a priori uncertain tasks*, whose processing times may deviate from its nominal values without any additional information;
- A set $\tilde{V}^2 \subseteq V$ of *a posteriori uncertain tasks*, whose uncertainty stems from their assignment to a specific subset of workstations denoted by $\widehat{W} \subseteq W$, referred to as *uncertain workstations*.

In this setting, any task assigned to a workstation in $\widehat{W}$ is considered *uncertain*, even if it is not originally in $\tilde{V}^1$. We define the complete set of *uncertain tasks* as $\tilde{V} = \tilde{V}^1 \cup \tilde{V}^2$. Accordingly, workstations in $W \setminus \widehat{W}$ are termed *certain*.

To evaluate the robustness of a feasible solution under task time uncertainty, the authors employ the notion of a *stability radius*. Before introducing its formal definition, we will introduce the following notations:

- $t = (t_1, t_2, \dots, t_n) \in \mathbb{R}_{\geq}^n$ represents the vector of nominal processing times for each task;¹
- $\Xi = \left\{ \xi \in \mathbb{R}_{\geq}^n \mid \xi_j = 0 \text{ for all } j \in V \setminus \tilde{V} \right\}$ is the set of allowable deviations vectors, i.e., deviations are only permitted for uncertain tasks;
- $F(t)$ denotes the set of all feasible task-to-workstation assignments under the nominal task time vector t .

The *stability radius* of a feasible solution $s \in F(t)$ measures the maximum level of uncertainty under which the solution remains valid. Formally, it is defined as [3]:

$$\rho(s, t) = \max \{ \epsilon \geq 0 \mid \forall \xi \in B(\epsilon), s \in F(t + \xi) \}, \quad (2.5)$$

¹ $\mathbb{R}_{\geq}^n$ denotes the set of n -dimensional vectors with non-negative real components.

where the *stability ball* $B(\epsilon)$ is defined by $B(\epsilon) = \{\xi \in \Xi \mid \|\xi\| \leq \epsilon\}$.

In other words, the stability radius $\rho(s, t)$ corresponds to the largest value $\epsilon \geq 0$ for which the solution s remains feasible under all deviations ξ satisfying $\|\xi\| \leq \epsilon$.

In our work in this report, we are focusing only in one of the cases mentioned in [3]. We will consider the ℓ_∞ -norm: $\|\xi\|_\infty = \max_{j \in \tilde{V}} \xi_j$. As a consequence, we denote the corresponding stability radius and stability ball by $\rho_\infty(\cdot, \cdot)$ and $B_\infty(\cdot)$ for the ℓ_∞ -norm.

We will start by studying the following two illustrative examples to interpret the stability radius under the ℓ_∞ -norm, where the first one is the same one studied in [3].

Example 2.1 Consider the following problem instance: $n = 6$, $m = 2$, $\tilde{V}^1 = \{1, 2\}$, $t = (1, 1, 1, 1, 1, 1)$, $T = 4$, and $\widehat{W} = \emptyset$. There are no precedence constraints. Two feasible solutions s' and s'' where $V_1^{s'} = (3, 4, 1)$, $V_2^{s'} = (5, 6, 2)$, $V_1^{s''} = (1, 2)$, $V_2^{s''} = (3, 4, 5, 6)$. These two solution are shown in the Figure 2.1 respectively.

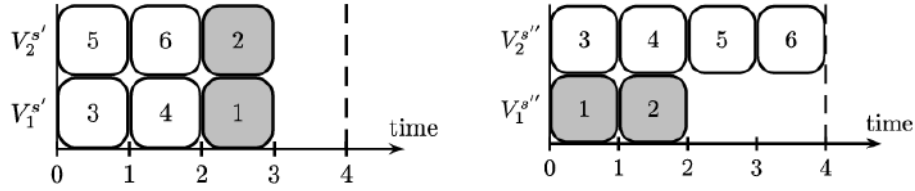


Figure 2.1: Solutions s' and s'' for the instance in Example 1.

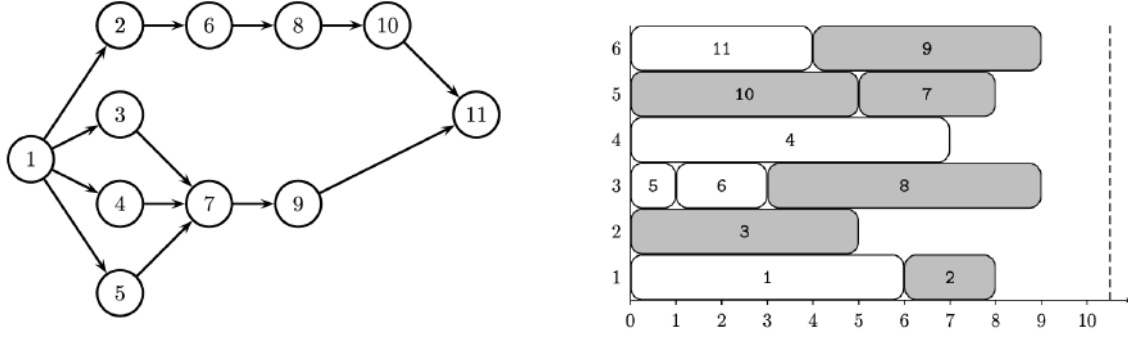
It can be observed that increasing the processing time of any uncertain task by at most one unit does not affect the feasibility of the solutions s' and s'' . In other words, for all deviation vectors $\xi \in B_\infty(1)$, both solutions remain feasible, i.e., $s' \in F(t + \xi)$ and $s'' \in F(t + \xi)$. However, this feasibility is no longer guaranteed if the deviation slightly exceeds the threshold. Specifically, for any $\delta > 0$, consider the deviation vector $\xi^* := (1 + \delta, 1, 0, 0, 0, 0) \in B_\infty(1 + \delta)$. Under this deviation, both s' and s'' become infeasible, meaning $s', s'' \notin F(t + \xi^*)$. Therefore, the stability radius with respect to the ℓ_∞ -norm for both solutions: $\rho_\infty(s', t) = \rho_\infty(s'', t) = 1$.

Example 2.2 Consider the instance called *JACKSON* instance: $n = 11$, $m = 6$, $\tilde{V}^1 = \{2, 3, 7, 8, 9, 10\}$, $T = 10.5$, and $\widehat{W} = \emptyset$. The precedence constraints are illustrated in Figure 2.2a. Finally, the feasible solution we will study are described in Figure 2.2b and the nominal task times for tasks 1 to 11 are given by the vector:

$$t = (6, 2, 5, 7, 1, 2, 3, 6, 5, 5, 4).$$

It can be observed that increasing the processing time of any uncertain task by at most 1.25 units does not affect the feasibility of the solution described in Figure 2.2b. In other words, for all deviation vectors $\xi \in B_\infty(1.25)$, the solution remains feasible. However, this feasibility is no longer guaranteed if the deviation slightly exceeds the threshold. Specifically, for any $\delta > 0$, consider the deviation vector:

$$\xi^* := (0, 1.25, 1.25, 0, 0, 0, 1.25 + \delta, 1.25, 1.25, 1.25, 0) \in B_\infty(1.25 + \delta).$$



(a) Precedence constraints of the JACKSON instance

(b) Feasible solution of JACKSON with $\rho_\infty = 1.25$

Figure 2.2: Precedence constraints and feasible solution for JACKSON Instance.

Under this deviation, the solution described in Figure 2.2b becomes infeasible. Therefore, the stability radius of the solution with respect to the ℓ_∞ -norm is 1.25.

The above two examples illustrate how to calculate the stability of the radius ρ_∞ given a feasible solution (feasible assignment of tasks to workstations). But in [3], A. Rossi et al. did not limit their contribution for using the concept of the stability radius to calculate the stability of the radius ρ_∞ . They also proved that, for the ℓ_∞ -norm, the stability radius can be computed using the following theorem.

Theorem 2.1 *The stability radius $\rho_\infty(s, t)$ for $s \in F(t)$ is calculated as follows:*

$$\rho_\infty(s, t) = \min_{k \in \widetilde{W}^s} \frac{T - \sum_{j \in V_k^s} t_j}{|\widetilde{V}_k^s|}. \quad (2.6)$$

where $\widetilde{W}^s = \{k \in W \mid \widetilde{V}_k^s \neq \emptyset\}$, $\widetilde{V}_k^s = V_k^s \cap \widetilde{V}$.

Theorem 2.1, states that to calculate the stability of the radius ρ_∞ , we seek for the workstation that provides the minimum value of the idle time divided by the number of its uncertain tasks.

Moreover, as discussed in the introduction, the authors of [3] also investigate the computational complexity of SALBP-S. They prove that the problem P_∞ , which corresponds to the stability radius maximization under the ℓ_∞ -norm, is strongly $\mathcal{NP}$ -hard. In addition to this complexity result, they propose a tight theoretical upper bound UB_∞ for P_∞ . The details of how this upper bound is derived and computed can be found in Section 3.2 of the same paper.

Finally, the authors also propose the following Mixed-Integer Linear Programming (MILP) formulation for P_∞ . In this formulation, ρ_∞ represents the stability radius to be maximized. The binary variable $x_{j,k}$ is equal to one if and only if task j is assigned to workstation k . The variable $\xi_{j,k}$ denotes the deviation in the processing time of task j on workstation k .

$$\begin{aligned} & \text{Maximize } \rho_\infty \\ & \sum_{k \in W} x_{j,k} = 1 \quad \forall j \in V \end{aligned} \quad (2.7)$$

$$\xi_{j,k} \leq UB_{\infty} x_{j,k} \quad \forall j \in V, \forall k \in W \quad (2.8)$$

$$\rho_{\infty} = \sum_{k \in W} \xi_{j,k} \quad \forall j \in V \quad (2.9)$$

$$\sum_{j \in V} t_j x_{j,k} + \sum_{j \in V} \xi_{j,k} \leq T \quad \forall k \in \widehat{W} \quad (2.10)$$

$$\sum_{j \in V} t_j x_{j,k} + \sum_{j \in \widetilde{V}^1} \xi_{j,k} \leq T \quad \forall k \in W \setminus \widehat{W} \quad (2.11)$$

$$\sum_{k \in W} k x_{i,k} \leq \sum_{k \in W} k x_{j,k} \quad \forall (i,j) \in A \quad (2.12)$$

$$\sum_{q=k}^m x_{i,q} \leq \sum_{q=k}^m x_{j,q} \quad \forall (i,j) \in A, \forall k \in W \setminus \{1\} \quad (2.13)$$

$$x_{j,k} = 0 \quad \forall j \in V, \forall k \notin Q(j) \quad (2.14)$$

$$\rho_{\infty} \geq 0$$

$$\xi_{j,k} \geq 0 \quad \forall j \in V, \forall k \in W$$

$$x_{j,k} \in \{0, 1\} \quad \forall j \in V, \forall k \in W$$

with

$$Q(j) = \left[\left\lceil \frac{t_j + \sum_{i \in \mathcal{P}(j)} t_i}{T} \right\rceil, m+1 - \left\lfloor \frac{t_j + \sum_{i \in \mathcal{S}(j)} t_i}{T} \right\rfloor \right].$$

where $\mathcal{P}(j)$ (resp. $\mathcal{S}(j)$) is a set of all predecessors (resp. all successors) of j in the graph G representing the precedence constraints.

Constraint (2.7) in the above formulation, ensures that each task is allocated to exactly one work station. While constraints (2.8) and (2.9), ensures that the maximum deviation in the processing time of task j on workstation k will be the upper bound UB_{∞} if task j is assigned to the workstation k and the deviation will be set to zero otherwise. Moreover, since the task j is assigned to exactly one workstation (say k^*) from constraint (2.7), then the only non-zero $\xi_{j,k}$ corresponds to the assigned workstation from constraint (2.8). Thus, the stability of the radius $\rho_{\infty} = \xi_{j,k^*}$.

Constraints (2.10) and (2.11) states that the total load of each workstation does not exceed the fixed cycle time T , whatever possible processing task time deviations. Not only that, but they also indicate that the deviation $\xi_{j,k}$ has no impact if the workstation k is certain ($k \in \widehat{W}$) and the task j is certain ($j \in V \setminus \widetilde{V}$). Constraints (2.12) and (2.13) express the precedence constraints. Finally, (2.14) induce that the task j can only be allocated to a restricted set of workstations denoted by the interval $Q(j)$. This is very important constraint as it reduces the search space.

As mentioned earlier, the problem P_{∞} has been proven to be strongly $\mathcal{NP}$ -hard in [3]. In this report, we aim to solve it efficiently by employing Benders Decomposition along with other complementary techniques. Before applying Benders Decomposition to our MILP formulation, we begin by providing a detailed description of the method in the following section. We will then proceed to apply it to our problem.

2.2 Benders Decomposition

Benders decomposition[9, 10] is a solution method for solving certain large-scale optimization problems. Instead of considering all decision variables and constraints of a large-scale problem simultaneously, Benders decomposition partitions the problem into multiple smaller problems. Since computational complexity of optimization problems increases significantly with the number of variables and constraints, solving these smaller problems iteratively can be more efficient than solving a single large problem. In this section, we will describe Benders decomposition for solving linear programs.

Consider the following Mixed-Integer Linear Programming (MILP) problem:

$$\begin{aligned} & \text{Maximize } c^T x + f^T y \\ & \text{subject to: } Ax + By \leq b \\ & \quad x \geq 0; \\ & \quad y \in Y. \end{aligned} \tag{2.15}$$

where Y is a polyhedral set, A, B are matrices, while b, c, f are vectors of compatible dimensions. We assume that the y -variables are considered *complicating variables*, meaning that the problem becomes significantly easier to solve when these variables are fixed.

To handle this structure, Benders decomposition is employed. It separates the original problem (2.15) into two interconnected problems:

- *Master problem*: involving only the complicating variables y ,
- *Subproblem*: involving the remaining variables x .

It is straightforward to observe that problem (2.15) can be equivalently reformulated as a two-level optimization problem:

$$\text{Maximize}_{y \in Y} \left\{ f^T y + \text{Maximize} \left\{ c^T x \left| \begin{array}{l} Ax \leq b - By \\ x \geq 0 \end{array} \right. \right\} \right\} \tag{2.16}$$

The inner maximization problem is a linear program for any given value of $y \in Y$. If this inner problem is unbounded for some $y \in Y$, then the entire problem (2.15) is also unbounded. Therefore, we will assume that for all $y \in Y$, the inner problem is bounded.

Given this assumption, we can evaluate the inner problem's optimal value by solving its dual. Let α be the vector of dual variables associated with the constraints $Ax \leq b - By$. Then, the dual of the inner problem is given by:

$$\begin{aligned} & \text{Minimize } \alpha^T (b - By) \\ & \text{subject to: } A^T \alpha \geq c, \\ & \quad \alpha \geq 0. \end{aligned} \tag{2.17}$$

Notice that the feasible region of the dual problem (2.17) does not depend on the value of y ; only the objective function is affected by y . Thus, if the dual feasible region is empty, then either:

- The primal inner problem is unbounded for some $y \in Y$, implying that the original MILP (2.15) is also unbounded, or

- The primal feasible region is empty for all $y \in Y$, making the original MILP (2.15) infeasible as well.

Assuming that the feasible region of the dual problem (2.17) is non-empty, we can apply Minkowski's Theorem to represent this polyhedral region as the convex combination of its extreme points $(\alpha_p^1, \dots, \alpha_p^I)$ and the positive combination of its extreme rays $(\alpha_r^1, \dots, \alpha_r^J)$.

For any fixed vector $\bar{y} \in Y$, the dual objective can be evaluated by examining:

- Whether there exists an extreme ray α_r^j such that $(\alpha_r^j)^T(b - B\bar{y}) < 0$, in which case the dual problem is unbounded for the given $\bar{y}$, implying that the primal inner problem is infeasible and hence the original problem (2.15) is also infeasible for that $\bar{y}$.
- Or, finding an extreme point α_p^i that minimizes the value of the objective function $(\alpha_p^i)^T(b - B\bar{y})$, in which case both primal and dual formulations have finite optimal solutions.

This leads to the following reformulation of the dual problem:

$$\begin{aligned}
& \text{Minimize} && t \\
& \text{subject to:} && (\alpha_r^j)^T(b - By) \geq 0 \quad \forall j = 1, \dots, J \\
& && (\alpha_p^i)^T(b - By) \geq t \quad \forall i = 1, \dots, I \\
& && t \in \mathbb{R}.
\end{aligned} \tag{2.18}$$

Now, depending on the above reformulation (2.18) of the dual problem, the original MILP (2.15) can be reformulated as:

$$\begin{aligned}
& \text{Maximize} && f^T y + t \\
& \text{subject to:} && (\alpha_r^j)^T(b - By) \geq 0 \quad \forall j = 1, \dots, J \\
& && (\alpha_p^i)^T(b - By) \geq t \quad \forall i = 1, \dots, I \\
& && t \in \mathbb{R}; \\
& && y \in Y.
\end{aligned} \tag{2.19}$$

Since there is typically an exponential number of extreme points and extreme rays in the dual formulation (2.17), generating all the constraints presented in (2.19) is not practical. Instead, the Benders decomposition algorithm begins with a relaxed master problem that includes only a subset of these constraints. The algorithm proceeds iteratively as follows:

- Solve the relaxed master problem to obtain a candidate solution $(\bar{y}, \bar{t})$.
- Fix $\bar{y}$ and solve the corresponding dual subproblem (2.17).
 - If the dual subproblem is unbounded, then a feasibility cut (corresponding to an extreme ray α_r^j in (2.19)) is generated and added to the relaxed master problem. These are known as *Benders feasibility cuts*, as they ensure that the primal subproblem remains feasible.
 - If the dual subproblem is feasible and bounded, we check if the subproblem objective value matches the value of the master problem objective:
 - If they match, the algorithm stops.

- otherwise, an optimality cut (corresponding to an extreme point α_p^i in (2.19)) is generated and added to the relaxed master problem. These are known as *Benders optimality cuts*, since they are derived from optimality conditions of the subproblem.
- The relaxed master problem is re-solved with the new cut, and the process repeats until the algorithm terminates (i.e. we reach the optimal solution of (2.15)).

So, each iteration refines the description of the feasible set of the master problem in the vicinity of its optimal solution by adding extreme rays or an extreme point.

Since the number of extreme points and rays (I and J) is finite, and a new cut is generated in each iteration, the algorithm is guaranteed to converge to an optimal solution in a finite number of steps.

All of the above describes how to apply Benders Decomposition to the MILP's, including its structure, the roles of the master and subproblems, and strategies for generating and injecting Benders cuts.

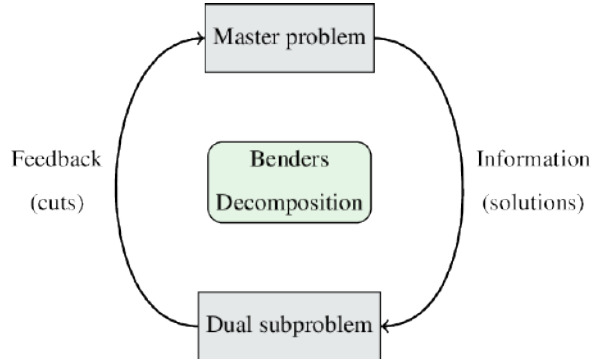


Figure 2.3: Benders Decomposition

In the following section, we apply Benders Decomposition to the MILP formulation designed for maximizing the stability radius. We detail how the problem can be decomposed, how the dual subproblem is constructed, and how optimality cuts are generated.

2.3 Benders Decomposition on MILP Formulation to Maximize the Stability Radius

In this section, we apply Benders Decomposition to efficiently solve the MILP formulation introduced earlier in Section 2.1. In our context, the complicating variables are $(x_{j,k})_{j \in V, k \in W}$. Thus, we first solve a master problem that includes only these variables, and then, after fixing their values, we solve the resulting subproblem as described below.

At the beginning, we will introduce our master problem which aims to maximize the variable τ which introduced as a provisional objective for the stability radius ρ_∞ . It acts as a placeholder for ρ_∞ during the decomposition process. The master problem maximizes τ while enforcing task assignments $(x_{j,k})_{j \in V, k \in W}$ that satisfy cycle time, precedence, and workstation feasibility constraints. After

solving the master problem, if τ doesn't match the objective value of our dual subproblem, which will introduced later, an optimality cut is generated and added to the master problem. These cuts iteratively adjust τ to align it with the achievable ρ_∞ .

The formulation of our master problem will be:

$$\begin{aligned} \text{Maximize } & \tau \\ \sum_{k \in W} x_{j,k} &= 1 & \forall j \in V \end{aligned} \quad (2.20)$$

$$\sum_{j \in V} t_j x_{j,k} \leq T \quad \forall k \in W \quad (2.21)$$

$$\sum_{k \in W} k x_{i,k} \leq \sum_{k \in W} k x_{j,k} \quad \forall (i, j) \in A \quad (2.22)$$

$$x_{j,k} = 0 \quad \forall j \in V, \forall k \notin Q(j) \quad (2.23)$$

$$\tau \leq UB_\infty \quad (2.24)$$

$$x_{j,k} \in \{0, 1\} \quad \forall j \in V, \forall k \in W$$

$$\tau \geq 0$$

In the above formulation, constraints (2.21) and (2.24) is essential, as (2.21) ensures that the total processing time assigned for any workstation does not exceed the cycle time T , and (2.24) to avoid unboundedness of our master problem.

We emphasize that this master problem includes only the complicating variables $(x_{j,k})_{j \in V, k \in W}$, and does not yet model the full robustness objective. The role of the master problem is to provide a feasible task assignment $(\bar{x}_{j,k})_{j \in V, k \in W}$ and value of $\bar{\tau}$. Then we will use them to solve the subproblem and generate cuts to the master problem if needed.

Our subproblem, which involves the variables $(\xi_{j,k})_{j \in V, k \in W}$ and aims to maximize the value of ρ_∞ , can be formulated as follows:

$$\begin{aligned} \text{Maximize } & \rho_\infty \\ \rho_\infty &\leq \sum_{k \in W} \xi_{j,k} & \forall j \in V \end{aligned} \quad (2.25)$$

$$\sum_{j \in V} \xi_{jk} \leq T - \sum_{j \in V} t_j x_{jk} \quad \forall k \in \widehat{W} \quad (2.26)$$

$$\sum_{j \in \widetilde{V}^1} \xi_{jk} \leq T - \sum_{j \in V} t_j x_{jk} \quad \forall k \in W \setminus \widehat{W} \quad (2.27)$$

$$\xi_{jk} \leq UB_\infty x_{jk} \quad \forall j \in V, \forall k \in W \quad (2.28)$$

$$\xi_{jk} \geq 0 \quad \forall j \in V, \forall k \in W$$

$$\rho_\infty \geq 0$$

To derive the dual of our subproblem, we assign the non-negative dual variables $(\alpha_j)_{j \in V}$, $(\beta_k)_{k \in \widehat{W}}$, $(\gamma_k)_{k \in W \setminus \widehat{W}}$, and $(\delta_{j,k})_{j \in V, k \in W}$ to the first, second, third and fourth primal constraints, respectively.

To simplify notation, we define for each workstation $k \in W$:

$$S_k = T - \sum_{j \in V} t_j x_{j,k}$$

Using all of the above, the dual formulation associated with the primal subproblem becomes:

$$\begin{aligned} \text{Minimize} \quad & \sum_{k \in \widehat{W}} \beta_k S_k + \sum_{k \in W \setminus \widehat{W}} \gamma_k S_k + UB_\infty \sum_{j \in V, k \in W} \delta_{jk} x_{jk} \\ & -\alpha_j + \beta_k + \delta_{jk} \geq 0 \quad \forall j \in V, \forall k \in \widehat{W} \end{aligned} \quad (2.29)$$

$$-\alpha_j + \gamma_k + \delta_{jk} \geq 0 \quad \forall j \in \widetilde{V}^1, \forall k \in W \setminus \widehat{W} \quad (2.30)$$

$$-\alpha_j + \delta_{jk} \geq 0 \quad \forall j \notin \widetilde{V}^1, \forall k \in W \setminus \widehat{W} \quad (2.31)$$

$$\sum_{j \in V} \alpha_j = 1 \quad (2.32)$$

$$\alpha_j, \delta_{jk} \geq 0 \quad \forall j \in V, \forall k \in W.$$

$$\beta_k \geq 0 \quad \forall k \in \widehat{W}$$

$$\gamma_k \geq 0 \quad \forall k \in W \setminus \widehat{W}$$

We now proceed with Benders decomposition. Initially, we solve the master problem to obtain a feasible assignment of the variables $(\bar{x}_{j,k})_{j \in V, k \in W}$ and a corresponding value $\bar{\tau}$. Given this assignment $(\bar{x}_{j,k})_{j \in V, k \in W}$, we then solve the dual subproblem.

The boundedness of the dual subproblem is guaranteed as the dual variables are constrained to be non-negative, and the quantities:

$$\bar{S}_k = T - \sum_{j \in V} t_j \bar{x}_{j,k}$$

for any workstation are strictly positive by construction from the master problem. Therefore, throughout the iterations, only optimality cuts are added to the master problem as the boundedness of the dual one is guaranteed.

These optimality cuts take the following form:

$$\sum_{k \in \widehat{W}} \bar{\beta}_k S_k + \sum_{k \in W \setminus \widehat{W}} \bar{\gamma}_k S_k + UB_\infty \sum_{j \in V, k \in W} \bar{\delta}_{j,k} x_{j,k} \geq \tau \quad (2.33)$$

where $(\bar{\beta}_k)_{k \in \widehat{W}}$, $(\bar{\gamma}_k)_{k \in W \setminus \widehat{W}}$, and $(\bar{\delta}_{j,k})_{j \in V, k \in W}$ are the dual variables obtained by solving the dual subproblem associated with the current assignment $(\bar{x}_{j,k})_{j \in V, k \in W}$.

The decomposition algorithm proceeds iteratively: the master problem provides a candidate assignment $(\bar{x}_{j,k})_{j \in V, k \in W}$ and a candidate value $\bar{\tau}$ for the objective. Based on this solution, we solve the dual subproblem. If the value of the master objective $\bar{\tau}$ equal to the optimal objective value of the dual subproblem, then the overall optimal solution has been reached. Otherwise, an optimality cut described in inequality (2.33) is added to the master problem for the next iteration.

2.3.1 Computational Results

A JACKSON instance, which was introduced previously in Example 2.2 is used to test our decomposition. As we said previously, this includes $n = 11$ tasks and $m = 6$ workstations. The cycle time for each

workstation is fixed at $T = 10.5$. Furthermore, the precedence constraints for this instance are illustrated in the Figure 2.2a in Section 2.1. While the nominal task times for tasks 1 to 11 are given by the vector:

$$t = (6, 2, 5, 7, 1, 2, 3, 6, 5, 5, 4).$$

In our experiments in this report, we assume that

$$|\widetilde{V}^1| \in \left\{0, \left\lceil \frac{n}{4} \right\rceil, \left\lceil \frac{n}{2} \right\rceil, n\right\}, \quad |\widehat{W}| \in \left\{0, \left\lceil \frac{m}{4} \right\rceil, \left\lceil \frac{m}{2} \right\rceil, m\right\}$$

where $\widetilde{V}^1$ (respectively $\widehat{W}$) is constructed by selecting the first $|\widetilde{V}^1|$ (respectively $|\widehat{W}|$) elements of a random permutation of $\{1, \dots, n\}$ (respectively $\{1, \dots, m\}$).

The random task permutation used is: (6, 4, 9, 8, 5, 1, 2, 7, 3) and the random workstation permutation is: (3, 2, 1, 5, 4).

In addition to the JACKSON instance, other instances such as MANSOOR (almost have the same size compared with JACKSON), MITCHELL (larger than JACKSON), and ROSZIEG (larger than JACKSON) were also used to evaluate the optimal value of the stability of the radius using our decomposition. Further details about these instances can be found in Appendix B.

The experiments were conducted on a computer equipped with an Intel® Core™ i3-N305 processor (1800 MHz) and 8 GB of RAM. Gurobi Optimizer version 12.0.1 was used as the solver for all computations in this report.

Tables 2.1, 2.2, 2.3, and 2.4 report the optimal values of the stability radius ρ_∞ obtained by applying Benders decomposition to our MILP formulation. Each table also includes the corresponding CPU time and the number of optimality cuts generated by the dual subproblem, which are iteratively added to the master problem to converge to the optimal solution ρ_∞ . These results are presented for every combination across the four instances: JACKSON, MANSOOR, MITCHELL, and ROSZIEG.

Our results align exactly with those reported in [3], where the authors used Xpress-Mosel to solve the MILP formulation. This confirms the correctness of our implementation. However, we observe that for some combinations in the MANSOOR, MITCHELL, and ROSZIEG instances, the optimal ρ_∞ is marked as dnf (did not found), and the corresponding CPU time exceeds 180 seconds. This indicates that our Benders decomposition approach was unable to reach the optimal value of ρ_∞ within the 3-minute time limit for those combinations.

Finally, we observe also from the tables that the number of optimality cuts added iteratively to the master problem increases as the instance size grows. Each time a new cut is added and a feasible solution to the master problem is found, the dual subproblem must be solved again. This repeated solving becomes increasingly time-consuming for larger instances.

Therefore, our next step in the next section, is to introduce a tailored algorithm for solving the dual subproblem more efficiently. Although standard MILP solvers like Gurobi can be used to solve the dual, our goal is to design a specialized algorithm that outperforms the solver in speed and contributes to reducing the overall runtime of the Benders decomposition process.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Optimal ρ_∞	1.50	5.50	1.50	1.25	1.25	0.75	0.75
CPU time (s)	0.21	0.19	0.32	0.81	0.33	0.22	0.51
Number of cuts	5	5	9	16	6	5	11

Table 2.1: Stability of radius ρ_∞ , CPU time (in seconds), number of optimality cuts add to the master problem in order to obtain the optimal solution for each combination of the **JACKSON** instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Optimal ρ_∞	21.50	67.50	dnf	12.75	23.75	7.75	dnf
CPU time (s)	0.43	0.10	>180	0.84	0.32	1.40	>180
Number of cuts	11	2	–	18	6	22	–

Table 2.2: Stability of radius ρ_∞ , CPU time (in seconds), number of optimality cuts add to the master problem in order to obtain the optimal solution for each combination of the **MANSOOR** instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Optimal ρ_∞	4.25	11.50	3.50	1.75	dnf	1.50	1.13
CPU time (s)	1.70	1.64	3.91	6.30	>180	7.10	4.44
Number of cuts	17	15	32	41	–	45	36

Table 2.3: Stability of radius ρ_∞ , CPU time (in seconds), number of optimality cuts add to the master problem in order to obtain the optimal solution for each combination of the **MITCHELL** instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Optimal ρ_∞	3.25	6.50	1.17	1.75	1.75	1.17	dnf
CPU time (s)	7.75	1.46	8.95	10.22	7.05	11.48	>180
Number of cuts	39	11	43	44	41	58	–

Table 2.4: Stability of radius ρ_∞ , CPU time (in seconds), number of optimality cuts add to the master problem in order to obtain the optimal solution for each combination of the **ROSZIEG** instance.

2.4 Tailored Algorithms for Solving the Dual subproblem

After applying Benders decomposition to our MILP formulation and testing it in the previous sections, we observed that the number of optimality cuts added to the master problem increases significantly as the instance size grows. Since the dual subproblem must be solved after each new cut is added and a feasible master solution is found, the dual is invoked many times throughout the Benders process. This repeated solving becomes increasingly costly for larger instances. Therefore, in this section, we propose a set of tailored algorithms specifically designed to accelerate the solving of the dual subproblem by exploiting the structure of the dual problem and leveraging information available from the master problem at each iteration.

Specifically, we treat the assignment variables $(\bar{x}_{j,k})_{j \in V, k \in W}$ from the master problem as fixed inputs and design four specialized algorithms to solve the dual subproblem more efficiently. These algorithms correspond to four distinct cases that arise based on the structure of the sets $\tilde{V}^1$, $\widehat{W}$, and W .

1. Case 1: The set of uncertain workstations is empty.

In this case, we assume that the set of uncertain workstations is empty, i.e., $\widehat{W} = \emptyset$. As a result, there are no β_k variables in the dual subproblem, and the dual objective function simplifies to:

$$\text{Minimize} \quad \sum_{k \in W} \gamma_k S_k + U B_\infty \sum_{j \in V, k \in W} \delta_{j,k} x_{j,k}$$

Here, constraint (2.29) is dropped from the dual formulation. Since the problem is a minimization subproblem, constraints (2.30) and (2.31) can be rewritten using the optimal relationships between variables as:

$$\gamma_k = \max_{j \in \tilde{V}^1} (\alpha_j - \delta_{j,k}, 0), \quad \forall k \in W \quad (2.34)$$

$$\alpha_j = \delta_{j,k}, \quad \forall j \notin \tilde{V}^1, \forall k \in W \quad (2.35)$$

Algorithm 1 for this case exhibits a computational complexity $\mathcal{O}(|\tilde{V}^1||W||V| + |V||W|^2)$, and proceeds as follows. We begin by setting $\delta_{j,k} = 0$ for each pair (j, k) such that $x_{j,k} = 1$. Then, using constraint (2.35), we deduce that for every $j \notin \tilde{V}^1$, the corresponding $\alpha_j = 0$.

Next, among the remaining α_j variables (i.e., those for $j \in \tilde{V}^1$), we assign the value 1 to a single α_j and set all others to 0, ensuring that the constraint $\sum_{j \in V} \alpha_j = 1$ is satisfied.

For every $\alpha_j = 0$, we set the corresponding $\delta_{j,k} = 0$ for all $k \in W$. For the unique $\alpha_j = 1$, we set $\delta_{j,k} = 1$ for all $k \in W$ except the one for which $x_{j,k} = 1$; this particular $\delta_{j,k}$ remains 0, as it was initialized.

We then identify the workstation index k for which $x_{j,k} = 1$ for the current j with $\alpha_j = 1$. We assign $\gamma_k = 1$ and set all other $\gamma_k = 0$. In this case, the value of the dual objective becomes S_k , which is the value associated with the unique non-zero γ_k .

This procedure is repeated for each $j \in \tilde{V}^1$. For each iteration, we set $\alpha_j = 1$ (and all other $\alpha_j = 0$), apply the same construction, and store the resulting value of the dual objective.

Among all computed objective values, we retain the minimum as the optimal value for this case and the dual variables will be the ones corresponding to the minimal value.

At this point, we cannot yet confirm whether the constructed solution is optimal, as it remains unclear whether the value of γ_k can take values strictly less than 1. To investigate this, we continue exploring all possible configurations that may produce a better objective value.

We examine whether there are multiple assignments $\alpha_j = 1$ across different iterations mentioned above that lead to the same index k with $\gamma_k = 1$. If no such multiple assignments are found, we retain the previously obtained solution as the optimal one.

However, if we find multiple instances where $\gamma_k = 1$ for the same index k , we proceed to refine the dual variables as follows. First, we count how many times $\gamma_k = 1$ appears for the same index k across these instances. We then update the value of γ_k by setting it to $1/(\text{number of appearances})$, while setting all other $\gamma_{k'} = 0$ for $k' \neq k$. Next, for each α_j that previously corresponded to this shared index k , we update its value by setting $\alpha_j = 1/(\text{number of appearances})$, and set all remaining $\alpha_j = 0$. To update the corresponding $\delta_{j,k}$ values, we use the updated α_j and γ_k values. For each non-zero α_j , we identify its index j , and for the corresponding non-zero γ_k , we take its index k . We then set $\delta_{j,k} = 1/(\text{number of appearances})$, except for the pair (j, k) where $x_{j,k} = 1$, which remains fixed at 0 (as initialized). All remaining $\delta_{j,k}$ values are set to 0.

The resulting objective value in this case is equal to $S_k \times \gamma_k$, where γ_k is the updated (fractional) value described above. After this refinement, we check whether there are any other indices k for which multiple $\gamma_k = 1$ assignments have occurred. If not, we compare the new objective value obtained from this refined configuration with the previous value from the initial construction, and keep the minimum of the two. The corresponding dual variables are then taken from the configuration that produced the minimum objective value. But if there are additional shared $\gamma_k = 1$ values for new indices k are found, we repeat the above refinement process for each of them, recomputing the objective and updating the dual variables accordingly. Ultimately, the final objective value will be the minimum among all configurations considered, and the dual variables retained are those that yield this minimal objective.

2. Case 2: The set of a priori uncertain tasks is empty.

In this case, we assume that the set of a priori uncertain tasks is empty, i.e., $\tilde{V}^1 = \emptyset$. As a result, there are no γ_k variables in the dual subproblem, and the dual objective function simplifies to:

$$\text{Minimize} \quad \sum_{k \in \widehat{W}} \beta_k S_k + U B_\infty \sum_{j \in V, k \in W} \delta_{j,k} \cdot x_{j,k}$$

Here, constraint (2.30) is dropped from the dual formulation. Since the problem is a minimization problem, constraints (2.29) and (2.31) can be rewritten using the optimal relationships between variables as follows:

$$\beta_k = \max_{j \in V} (\alpha_j - \delta_{j,k}, 0), \quad \forall k \in \widehat{W} \quad (2.36)$$

$$\alpha_j = \delta_{j,k}, \quad \forall j \notin \tilde{V}^1, \forall k \in W \setminus \widehat{W} \quad (2.37)$$

Algorithm 2 for this case exhibits a computational complexity of $\mathcal{O}(R|W||V| + |V||W|^2)$, where R denotes the number of active variables $x_{j,k}$ with $k \in W \setminus \widehat{W}$. It follows the same logic as in the previous case, with minor modifications. We begin by setting $\delta_{j,k} = 0$ for each pair (j, k) such that $x_{j,k} = 1$, as was done in Case 1.

Then, based on constraint (2.37), we inspect the variables $\delta_{j,k}$ corresponding to the entries where $x_{j,k} = 1$ with $k \in W \setminus \widehat{W}$. For each such pair, we set the corresponding $\alpha_j = 0$.

From this point onward, the algorithm proceeds in the same way as described in Case 1, except that we work with the β_k variables instead of γ_k . Full details are provided in Algorithm 2 at the end of this chapter.

3. Case 3: All workstations are uncertain.

In this case, we assume that all workstations are uncertain, i.e., $\widehat{W} = W$. As a result, there are no γ_k variables in the dual subproblem, and the dual objective function becomes the same as the one presented in Case 2. However, unlike Case 2, here both constraint (2.30) and constraint (2.31) are dropped from the formulation. The only remaining constraints are (2.32) and (2.29), where the latter can be rewritten using the optimal relationships between variables as described in equation (2.36).

Algorithm 3 for this case exhibits a computational complexity of $\mathcal{O}(|W||V|^2 + |V||W|^2)$, and follows the same general approach as in the previous cases. We begin by setting $\delta_{j,k} = 0$ for every pair (j, k) such that $x_{j,k} = 1$ in the input assignment.

The main difference in this case is that all α_j variables for $j \in V$ are now considered. Specifically, we iterate through each $j \in V$, assigning $\alpha_j = 1$ and setting all other $\alpha_{j'} = 0$ for $j' \neq j$. For each such assignment, we follow the same steps as described in Case 1 to construct the corresponding dual variables and compute the objective value.

This procedure is repeated for every $j \in V$, and after all iterations, we continue as in Case 1 to know whether these assignments give us the optimal value of the dual objective or we must continue. The complete process is detailed in Algorithm 3 at the end of this chapter.

4. Case 4: Both certain and uncertain workstations exist, and the set of a priori uncertain tasks is non-empty.

This is the final case we consider, in order to cover all possible combinations. Here, we assume that both the sets of certain and uncertain workstations are non-empty, and that the set of a priori uncertain tasks is also non-empty, i.e., $\widehat{W} \neq \emptyset$, $W \setminus \widehat{W} \neq \emptyset$, and $\widetilde{V}^1 \neq \emptyset$.

In this case, we consider the full formula of the dual objective function, and none of the constraints are dropped. The constraints (2.29), (2.30) and (2.31) are rewritten using the optimal relationships between variables as described in equations (2.34), (2.36), and (2.37), respectively.

Algorithm 4 for this case exhibits a computational complexity of $\mathcal{O}(R|W||V| + |V||W|^2)$, where $R = |V| - \text{number of active variables } x_{j,k} \text{ with } j \notin \widetilde{V}^1 \text{ and } k \in W \setminus \widehat{W}$. It starts by setting $\delta_{j,k} = 0$ for every pair (j, k) such that $x_{j,k} = 1$ in the input assignment. Then based on constraint (2.37), we inspect the variables $\delta_{j,k}$ corresponding to the entries where $x_{j,k} = 1$ with

$j \notin \tilde{V}^1$ and $k \in W \setminus \widehat{W}$. For each such pair, we set the corresponding $\alpha_j = 0$.

Next, among the remaining α_j variables not assigned to zero, we assign the value 1 to a single α_j and set all others to 0.

For every $\alpha_j = 0$, we set the corresponding $\delta_{j,k} = 0$ for all $k \in W$. For the unique $\alpha_j = 1$, we set $\delta_{j,k} = 1$ for all $k \in W$ except the one for which $x_{j,k} = 1$; this particular $\delta_{j,k}$ remains 0, as it was initialized.

We then identify the workstation index k for which $x_{j,k} = 1$ for the current j with $\alpha_j = 1$ and check whether this index k belongs to $\widehat{W}$ or $W \setminus \widehat{W}$. If $k \in \widehat{W}$, we set $\beta_k = 1$ and set all other $\beta_{k'}$ and $\gamma_{k'}$ (for $k' \neq k$) to zero. If instead $k \in W \setminus \widehat{W}$, we set $\gamma_k = 1$ and set all other $\beta_{k'}$ and $\gamma_{k'}$ (for $k' \neq k$) to zero.

The value of the dual objective in this configuration is S_k , where k corresponds to the index of the non-zero β_k or γ_k .

This procedure is repeated for each j from the remaining indices not initially assigned to zero. In each iteration, we set $\alpha_j = 1$ and all other $\alpha_{j'} = 0$ for $j' \neq j$, then apply the same construction to compute the corresponding dual variables and the value of the objective function.

Among all computed objective values, we retain the minimum as the optimal value for this case, and the corresponding dual variables are those associated with that minimum.

However, as in Case 1, we cannot yet confirm whether the constructed solution is optimal, as it remains unclear whether the values of β_k or γ_k can take values strictly less than 1. To investigate this, we continue exploring all possible configurations that may yield a better objective value, exactly as described in Case 1. However, in this case, we take into account that the dual subproblem includes not only γ_k variables but also β_k variables. The complete procedure is detailed in Algorithm 4 at the end of this chapter.

Remark 2.1 *Before proposing the algorithms described above, we initially developed a method based on the Projected Gradient Descent (PGD) algorithm. This approach iteratively updates the variables in the direction of the negative gradient and then projects the result back onto the feasible region. The full details of this algorithm are provided in Appendix A.*

After testing, we observed that this algorithm returns the same optimal values for the dual subproblem as those obtained by standard solvers. However, it turned out to be slower in practice. Additionally, the algorithm relies on a reduction parameter τ , which is used in the backtracking line search procedure guided by the Armijo–Goldstein condition. This condition helps determine an appropriate step size by starting with a large estimate and reducing it iteratively by the factor τ until the condition is satisfied.

The challenge, however, is that τ must be selected manually, typically by trial and error, as there is no known formula for computing an optimal value that ensures fast convergence. Due to these limitations, we decided not to rely on this algorithm and instead proposed alternative, more efficient methods as described above. Nonetheless, it is worth mentioning that if a principled or automated method for selecting τ can be developed, this algorithm may become competitive or even outperform

our proposed algorithms. More details can be found about this algorithm in Appendix A.

Remark 2.2 Before presenting the computational results related to our algorithms, it is important to highlight a key aspect that deepens our understanding of the proposed decomposition framework: the meaning of the Benders optimality cuts, specifically the inequality (2.33).

Up to this point, we have focused on how to construct these cuts using our tailored algorithms. However, we have been generating and adding them to the master problem without interpreting their structure or understanding their implications. This process, though effective, lacks insight into what the cuts truly represent.

Fortunately, the design of our algorithms offers an opportunity to interpret these optimality cuts more meaningfully. In what follows, we explain how these cuts can be understood through the lens of the algorithm's logic.

Let us denote by k^* the workstation that limits the stability radius. Then, inequality (2.33) can be expressed in different forms, depending on the specific configuration of the sets $\tilde{V}^1$, $\widehat{W}$, and W .

1. **Case 1: The set of uncertain workstations is empty.**

$$\tau \leq \frac{1}{\sum_{j \in \tilde{V}^1} \bar{x}_{j,k^*}} \left[\left[T - \sum_{j \in V} t_j x_{j,k^*} \right] + UB_\infty \sum_{k \in W \setminus \{k^*\}} \sum_{\substack{j \in \tilde{V}^1 \\ \bar{x}_{j,k^*}=1}} x_{j,k} \right] \quad (2.38)$$

2. **Case 2: The set of a priori uncertain tasks is empty.**

$$\tau \leq \frac{1}{\sum_{j \in V} \bar{x}_{j,k^*}} \left[\left[T - \sum_{j \in V} t_j x_{j,k^*} \right] + UB_\infty \sum_{k \in W \setminus \{k^*\}} \sum_{\substack{j \in V \\ \bar{x}_{j,k^*}=1}} x_{j,k} \right] \quad (2.39)$$

3. **Case 3: All workstations are uncertain.** (Exactly as case 2, inequality (2.39))

4. **Case 4: Both certain and uncertain workstations exist, and the set of a priori uncertain tasks are non-empty.**

(a) If the workstation responsible for limiting the stability radius is uncertain, i.e. $k^* \in \widehat{W}$. (Exactly as case 2, inequality (2.39))

(b) If the workstation responsible for limiting the stability radius is certain, i.e. $k^* \in W \setminus \widehat{W}$. (Exactly as case 1, inequality (2.38))

In the above four cases, it is worth noting that the second term inside the brackets in the simplified Benders optimality cuts (2.38) and (2.39) can be further simplified. Indeed, by applying constraint (2.7), we obtain:

$$\sum_{k \in W \setminus \{k^*\}} \sum_{\substack{j \in \tilde{V}^1 \\ \bar{x}_{j,k^*}=1}} x_{j,k} = \sum_{\substack{j \in \tilde{V}^1 \\ \bar{x}_{j,k^*}=1}} (1 - x_{j,k^*}),$$

and

$$\sum_{k \in W \setminus \{k^*\}} \sum_{\substack{j \in V \\ \bar{x}_{j,k^*}=1}} x_{j,k} = \sum_{\substack{j \in V \\ \bar{x}_{j,k^*}=1}} (1 - x_{j,k^*}).$$

2.4.1 Computational Results

In this section, we evaluate the performance of our four proposed algorithms. We use the same instances, **JACKSON** and **ROSZIEG**, that were previously employed to test our Benders decomposition approach. However, the current focus is on verifying the correctness and assessing the efficiency of our tailored algorithms.

To validate correctness, we first check whether our proposed algorithms return the same optimal value as a solver when solving the dual subproblem. For each combination in both instances, we fix a feasible task-to-workstation assignment obtained from the master problem as input to our algorithm. In addition, the value of UB_∞ corresponding to each combination is also provided as input to the algorithm, along with the nominal task time vector t and the constant cycle time T . Then, we solve the corresponding dual subproblem twice: once using a solver (Gurobi), and once using the appropriate algorithm from those proposed. In all tested cases, we found that the optimal solutions obtained by the solver and by our algorithms were identical.

To assess computational efficiency, we compare the CPU time required to solve the dual subproblem using both approaches. Tables 2.5 and 2.6 summarize the results. The tailored algorithms consistently outperform the LP solver in terms of computational time. Based on this observation, we adopt the proposed algorithms to solve the dual subproblem throughout the remainder of this report, replacing the use of the LP solver.

Combinations	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$	$ \tilde{V}^1 = 0$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$	$ \tilde{V}^1 = 0$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$	$ \tilde{V}^1 = n$
	$ \hat{W} = 0$	$ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \hat{W} = 0$	$ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \hat{W} = m$
First Approach	0.0393	0.0118	0.0120	0.0183	0.0122	0.0188	0.0158
Second Approach	0.0011	0.0002	0.0004	0.0025	0.0005	0.0005	0.0010

Table 2.5: CPU time (in seconds) for each combination of the **JACKSON** instance using LP solver (First Approach) and the tailored algorithms (Second Approach) for solving the dual problem.

Combinations	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$	$ \tilde{V}^1 = 0$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$	$ \tilde{V}^1 = 0$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$	$ \tilde{V}^1 = n$
	$ \hat{W} = 0$	$ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \hat{W} = 0$	$ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \hat{W} = m$
First Approach	0.0213	0.0189	0.0312	0.0196	0.0622	0.0338	0.0321
Second Approach	0.0016	0.0010	0.0015	0.0052	0.0018	0.0032	0.0068

Table 2.6: CPU time (in seconds) for each combination of the **ROSZIEG** instance using LP solver (First Approach) and the tailored algorithms (Second Approach) for solving the dual problem.

After proposing efficient algorithms to solve the dual subproblem, one can observe that, in each iteration of the Benders decomposition, a new cut is added to the master problem based on the dual solution. However, the master problem is then resolved from scratch in every iteration, without leveraging the fact that it was already solved in the previous step. This raises an important question,

which we address in the next section: how can we avoid resolving the master problem from scratch after each new cut is added?

2.5 Branch-and-Benders-Cut Algorithm on MILP Formulation to Maximize the Stability Radius

In classical Benders decomposition, we iteratively solve a master problem to obtain a candidate solution and an optimal objective value. This solution is then passed to a dual subproblem to generate optimality cuts (no feasibility cuts are required in our formulation). These cuts are added to the master problem, which is then resolved from scratch. This process repeats until the dual objective value less than or equal the master objective value.

However, resolving the master problem entirely at each iteration becomes computationally expensive, especially with high number of optimality cuts. To address this inefficiency, we implement the Branch-and-Benders-Cut (BBC) algorithm. It is a hybrid approach that integrates Benders decomposition directly into the branch-and-bound tree of a MIP solver using lazy constraint callbacks. Instead of resolving the master problem externally after each cut, BBC allows cuts to be added dynamically during the solution process, without restarting the solve. Here is how it works:

1. Solve the master problem with branch-and-bound. Whenever a node yields a feasible integer solution, pause branching at that node and invoke the lazy-constraint callback.
2. In the callback, fix this feasible integer solution and solve the dual subproblem.
3. If the Benders optimality condition holds, accept the solution at this node (no cut is added), and let the solver continue the search as usual.
4. Otherwise, generate the corresponding Benders optimality cut (from the dual extreme point) and add it as a *lazy* constraint (globally) to the master problem; the current node is cut off (i.e., the candidate solution at this node is rejected, and the solver proceeds under the augmented model).
5. Resume tree exploration; all subsequent nodes are processed under the accumulated Benders cuts.

2.5.1 Computational Results

We will use the same four instances JACKSON, MITCHELL, and ROSZIEG that were employed in Subection 2.3.1 to evaluate the stability radius of our model. In this section, we compare the stability radius results obtained using the following three approaches:

- Benders Decomposition without the tailored algorithm for solving the dual subproblem,
- Branch-and-Benders-Cut (BBC) without the tailored algorithm for solving the dual subproblem,
- Branch-and-Benders-Cut (BBC) with the tailored algorithm for solving the dual subproblem.

Additionally, we compare the CPU time required by each method across the seven combinations in each instance. The results show that all three approaches yield the same value for the stability

radius. Tables 2.7, 2.8 and 2.9 reports the CPU time in seconds where the BBC approach using the tailored dual algorithm achieves the best computational performance, requiring significantly less CPU time across all instances.

Combinations	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
First Approach	0.21	0.19	0.32	0.81	0.33	0.22	0.51
Second Approach	0.15	0.12	0.18	0.21	0.11	0.20	0.22
Third Approach	0.10	0.06	0.09	0.16	0.05	0.09	0.09

Table 2.7: CPU time (in seconds) for each combination of the **JACKSON** instance using Benders decomposition without tailored algorithm, (BBC) without the tailored algorithm, (BBC) with the tailored algorithm for solving the dual subproblem.

Combinations	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
First Approach	1.70	1.64	3.91	6.30	>180	7.10	4.44
Second Approach	0.57	0.53	0.89	0.70	0.74	1.00	1.00
Third Approach	0.23	0.29	0.31	0.33	0.28	0.48	0.43

Table 2.8: CPU time (in seconds) for each combination of the **MITCHELL** instance using Benders decomposition without tailored algorithm, (BBC) without the tailored algorithm, (BBC) with the tailored algorithm for solving the dual subproblem.

Combinations	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
First Approach	7.75	1.46	8.95	10.22	7.05	11.48	>180
Second Approach	0.85	0.42	1.21	0.91	1.18	0.95	2.01
Third Approach	0.48	0.35	0.32	0.49	0.55	0.50	1.00

Table 2.9: CPU time (in seconds) for each combination of the **ROSZIEG** instance using Benders decomposition without tailored algorithm, (BBC) without the tailored algorithm, (BBC) with the tailored algorithm for solving the dual subproblem.

Up to this point, the results presented in the previous sections show that applying the Branch-and-Benders-Cut (BBC) algorithm combined with the tailored algorithms we proposed to our original MILP formulation yields the lowest observed CPU time among the tested approaches.

A natural question that arises is whether we can do even better. Specifically, is it possible to further reduce the computational time beyond what has been achieved using BBC algorithm? The following sections aim to explore and answer this question.

2.6 Improving the Efficiency of the Branch-and-Benders-Cut Algorithm

In this section, we aim to improve the efficiency of the Branch-and-Benders-Cut (BBC) algorithm. We begin in Subsection 2.6.1 by incorporating a heuristic algorithm into the BBC framework. However, as we will observe, this approach proves to be inefficient in practice. A detailed analysis of its performance is presented in Subsubsection 2.6.1.1, which leads us to explore an alternative strategy. Specifically, in Subsection 2.6.2, we propose a reformulation of the master problem based on the heuristic algorithm. The analysis in Subsubsection 2.6.2.1 shows that this reformulation will enhance the overall efficiency of the BBC algorithm.

2.6.1 Branch-and-Benders-Cut Algorithm with Pre-injected Cuts from a Heuristic Algorithm

In this subsection, we attempt to enhance the performance of the BBC algorithm by incorporating a heuristic algorithm. This heuristic is used to generate a set of feasible solutions of increasing robustness quality, discarding solutions whose stability radius is worse than previously found solutions. For each generated solution from this heuristic, we solve the corresponding dual subproblem by our proposed algorithms presented in Section 2.4 and produce a Benders cut, which is then added to the master problem. All such cuts are pre-injected before the BBC algorithm is executed.

The core idea of this algorithm is inspired by the work of [8]. However, we have modified it and introduced additional conditions to better adapt it to requirements of our problem. At the beginning a feasible solution contains only one empty workstation. The heuristic assigns tasks to this workstation until no task can be added during the capacity constraint of this workstation. Then we open a new workstation and becomes current. This continues until all tasks are assigned, or the current solution is found to be infeasible.

So, we start with workstation $k = 1$, initially empty and we initialize the value of $\rho_{int} = 0$ to accept initial solutions and progressively increase it. To choose a task to be assigned to workstation k , we will build a Candidate List $\mathcal{CL}(k)$ of tasks that can currently be assigned to workstation k . A task j is added to this list if and only if all these conditions hold:

1. Task j has not yet been assigned to any workstation.
2. All predecessors of task j have been assigned to a workstation $\leq k$.
3. Workstation $k \in Q(j)$. If $k \notin Q(j)$, do not add task j to the list.

4. Adding task j to workstation k does not violate the cycle time:

$$\sum_{j' \in V_k^s} t_{j'} + t_j \leq T$$

5. If $(V_k \cup \{j\}) \cap \tilde{V} \neq \emptyset$ the following inequality must holds:

$$\frac{T - \sum_{j' \in V_k^s} t_{j'} - t_j}{|\tilde{V}_k|} > \rho_{int}$$

If this condition fails, task j is not eligible for workstation k .

After that, we check the candidate list $\mathcal{CL}(k)$:

- If $\mathcal{CL}(k) \neq \emptyset$ then we randomly select one task j from the candidate list. Assign task j to workstation k . Recalculate the candidate list $\mathcal{CL}(k)$.
- If $\mathcal{CL}(k) = \emptyset$ then no more tasks can be feasibly assigned to workstation k . We open the next workstation $k = k + 1$, and build a new candidate list.

Repeat the above steps until all tasks are assigned. At this point, we have a feasible solution s . We evaluate its overall stability radius using:

$$\rho_\infty = \min_{k \in \tilde{W}} \frac{T - \sum_{j \in V_k} t_j}{|\tilde{V}_k|}$$

- If $\rho_\infty \leq \rho_{int}$: this solution doesn't improve the robustness, discard it.
- If $\rho_\infty > \rho_{int}$: this solution improves the robustness:
 - Update $\rho_{int} \leftarrow \rho_\infty$.
 - Save the current solution as the best solution found so far.

Finally, we repeat the entire procedure again and generate multiple feasible solutions, but only we keep those whose stability radius is strictly better than the one found before. For each of these improved solutions, we solve the corresponding dual subproblem and generate Benders cut, which is then added to the master problem. All such cuts are pre-injected before the BBC algorithm is executed.

In order to assess the effectiveness of this approach and to analyze the performance of the heuristic in more detail, the following Subsubsection 2.6.1.1 presents computational results obtained by applying this method. These results aim to provide a deeper understanding of its impact when combined with the BBC algorithm.

2.6.1.1 Computational Results

An instance called **SAWYER30** was used to conduct our experiments, where in this instance, a set of $n = 30$ tasks and $m = 11$ workstations are considered. The cycle time for each workstation is fixed at $T = 37.5$. More details about this instance can be found in Appendix B.

The results are summarized in the following tables:

- Table 2.10 reports the CPU time required by the heuristic algorithm to produce feasible solutions with increasing robustness, based on 100 independent runs for each combination of **SAWYER30** instance. Moreover, it presents the number of cuts added to the master problem using the solutions obtained from the same 100 heuristic runs in each combination of **SAWYER30** instance.
- Table 2.11 shows the number of lazy constraints added to the master problem during the execution of the BBC algorithm, with and without the heuristic algorithm, for the same seven combinations of **SAWYER30** instance.
- Table 2.12 compare the CPU times required by BBC algorithm to reach the optimal solution with and without the use of the heuristic algorithm, across seven combinations of **SAWYER30** instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Heuristic CPU time	0.08	0.07	0.08	0.11	0.09	0.12	0.06
Number of Cuts	10	8	9	8	9	9	10

Table 2.10: Heuristic CPU time (in seconds) for 100 run to produce feasible solutions of increasing robustness quality, the number of cuts added to the master problem by using the heuristic algorithm, for seven combinations of **SAWYER30** instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Without Heuristic	64	63	532	121	713	980	809
With Heuristic	43	49	558	107	815	1026	801

Table 2.11: Number of lazy constraints added to the master problem during BBC algorithm with and without the heuristic algorithm for seven combinations of **SAWYER30** instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Without Heuristic	3.8	1.8	73.7	4.3	45.1	92.8	180.8
With Heuristic	2.6	2.1	70.1	4.8	56.9	90.9	159

Table 2.12: CPU time (in seconds) required to reach the optimal solution by applying BBC algorithm with and without the heuristic algorithm for seven combinations of **SAWYER30** instance.

As shown in Table 2.10, the heuristic algorithm demonstrates strong efficiency, requiring minimal CPU time to generate feasible solutions. While Tables 2.11 and 2.12, show that the CPU time and

the number of lazy constraints did not decrease when applying the BBC algorithm with the heuristic, compared to the case without it. To further analyze the performance of our heuristic implementation and to understand why the addition of these cuts may be inefficient, we will present the solutions returned by the heuristic and compares the best stability radius achieved by the heuristic with that obtained by the BBC algorithm (optimal solution).

1. $|\tilde{V}^1| = \lceil \frac{n}{4} \rceil$, $|\widehat{W}| = 0$ with optimal $\rho_\infty = 9.5$
 - Solutions returned by heuristic: 0.5/ 1.25/ 1.75/ 2.5/ 3.25/ 4.5/ 5.5/ 5.75/ 6.5/ 7.5
2. $|\tilde{V}^1| = 0$, $|\widehat{W}| = \lceil \frac{m}{4} \rceil$ with optimal $\rho_\infty = 23.5$
 - Solutions returned by heuristic: 0.5/ 0.625/ 0.83/ 0.875/ 1.83/ 2.16/ 2.25/ 2.5
3. $|\tilde{V}^1| = \lceil \frac{n}{4} \rceil$, $|\widehat{W}| = \lceil \frac{m}{4} \rceil$ with optimal $\rho_\infty = 7.25$
 - Solutions returned the heuristic: 0.42/ 0.9/ 1.25/ 1.37/ 1.5/ 1.6/ 2.5/ 3.5/ 4.25
4. $|\tilde{V}^1| = \lceil \frac{n}{2} \rceil$, $|\widehat{W}| = 0$ with optimal $\rho_\infty = 5.25$
 - Solutions returned by heuristic: 0.25/ 1.25/ 1.75/ 2.25/ 2.75/ 3.25/ 3.5/ 3.75
5. $|\tilde{V}^1| = 0$, $|\widehat{W}| = \lceil \frac{m}{2} \rceil$ with optimal $\rho_\infty = 6.25$
 - Solutions returned by the heuristic: 0.125/ 0.3/ 0.6/ 0.8/ 0.9/ 1.16/ 1.625/ 2.16/ 2.375
6. $|\tilde{V}^1| = \lceil \frac{n}{2} \rceil$, $|\widehat{W}| = \lceil \frac{m}{2} \rceil$ with optimal $\rho_\infty = 4.17$
 - Solutions returned by heuristic: 0.38/ 0.83/ 1.13/ 1.16/ 1.38/ 1.5/ 2.16/ 2.25/ 2.38
7. $|\tilde{V}^1| = n$, $|\widehat{W}| = m$ with optimal $\rho_\infty = 2.5$
 - Solutions returned by heuristic: 0.16/ 0.38/ 0.5/ 0.87/ 1.16/ 1.25/ 1.3/ 1.5/ 1.75/ 1.83

However, this information alone is not sufficient to fully understand the behavior of the solutions returned by the heuristic. To gain deeper insights, we select two combinations from the above ones for detailed analysis. For each one, we report the optimal stability radius along with the index of the workstation responsible for it, followed by the set of tasks assigned to that workstation. The same information is then provided for all solutions generated by the heuristic algorithm. Additionally, we present the stability radius and the corresponding critical workstation for the first master solution evaluated during the callback.

We use the notation $(\rho_\infty, (3 : 4, 5, 6))$, which indicates that workstation 3 is responsible for the stability radius ρ_∞ , and tasks 4, 5, and 6 is the tasks assigned to this workstation.

1. $\tilde{V}^1 = [30, 3, 2, 9, 22, 12, 11, 17]$, $\widehat{W} = \emptyset$, $(|\tilde{V}^1| = \lceil \frac{n}{4} \rceil, |\widehat{W}| = 0)$
 - (a) Optimal $\rho_\infty = 9.5$ with $(7 : 7, 22)$
 - (b) Solutions returned by the heuristic algorithm:
 - i. $\rho_\infty^{(1)} = 0.50$ with $(1 : 3, 16, 1)$
 - ii. $\rho_\infty^{(2)} = 1.25$ with $(1 : 10, 3, 17, 16)$
 - iii. $\rho_\infty^{(3)} = 1.75$ with $(4 : 16, 9, 14, 15, 17)$
 - iv. $\rho_\infty^{(4)} = 2.50$ with $(8 : 21, 22)$
 - v. $\rho_\infty^{(5)} = 3.25$ with $(4 : 12, 8, 9, 13)$

- vi. $\rho_{\infty}^{(6)} = 4.50$ with $(1 : 3, 10, 1, 5)$
- vii. $\rho_{\infty}^{(7)} = 5.50$ with $(3 : 17, 6, 7, 18)$
- viii. $\rho_{\infty}^{(8)} = 5.75$ with $(1 : 10, 2, 12)$
- ix. $\rho_{\infty}^{(9)} = 6.50$ with $(1 : 10, 3, 1)$
- x. $\rho_{\infty}^{(10)} = 7.50$ with $(7 : 22, 23)$
- (c) First master solution evaluated by callback:
 - i. τ (master objective) = 17.7, $UB_{\infty} = 18.5$.
 - ii. Stability radius (dual objective) = 0.25 with $(11 : 11, 28, 29, 30)$
- 2. $\tilde{V}^1 = [30, 3, 2, 9, 22, 12, 11, 17, 29, 19, 27, 26, 16, 28, 23]$, $\widehat{W} = \emptyset$, $(|\tilde{V}^1| = \lceil \frac{n}{2} \rceil, |\widehat{W}| = 0)$
 - (a) Optimal $\rho_{\infty} = 5.25$ with $(2 : 2, 12, 13)$
 - (b) Solutions returned by the heuristic algorithm:
 - i. $\rho_{\infty}^{(1)} = 0.25$ with $(1 : 3, 1, 16)$
 - ii. $\rho_{\infty}^{(2)} = 1.25$ with $(3 : 3, 17, 7)$
 - iii. $\rho_{\infty}^{(3)} = 1.75$ with $(1 : 3, 2, 1)$
 - iv. $\rho_{\infty}^{(4)} = 2.25$ with $(2 : 11, 3)$
 - v. $\rho_{\infty}^{(5)} = 2.75$ with $(4 : 19, 11)$
 - vi. $\rho_{\infty}^{(6)} = 3.25$ with $(1 : 3, 1, 17, 5)$
 - vii. $\rho_{\infty}^{(7)} = 3.50$ with $(6 : 15, 7, 8, 9)$
 - viii. $\rho_{\infty}^{(8)} = 3.75$ with $(9 : 22, 23)$
 - (c) First master solution evaluated by callback:
 - i. τ (master objective) = 12.42, $UB_{\infty} = 12.42$.
 - ii. Stability radius (dual objective) = 1.17 with $(11 : 19, 29, 39)$

From the above results, if we take the optimal solution in the first case, tasks 7 and 22 are allocated to workstation 7, $t_7 + t_{22} = 14 + 14 = 28$, 22 only is uncertain, so we can check that the stability radius value for this workstation is $\frac{T-28}{1} = 9.5$. Similarly, for the optimal solution in the second case and the solutions returned by the heuristic algorithm.

Also, from the first case above, we observe that the value of $\tau = 17.7$ - which serves as a provisional objective for the stability radius ρ_{∞} in the master problem - when the first integer solution is found during the callback, is very close to the upper bound $UB_{\infty} = 18.5$. However, the optimal stability radius is significantly lower $\rho_{\infty} = 9.5$. This indicates that the inequalities generated before the Branch-and-Benders-Cut (BBC) procedure, based on heuristic solutions, are not efficient. Moreover, we note that the first integer solution found by the master problem corresponds to a stability radius of only 0.25, which is remarkably low. All of this suggests a natural way described in Subsection 2.6.2 to prevent the master problem from proposing solutions that are worse than the best solution found by the heuristic (*i.e.*, 7.5 in this case).

2.6.2 Branch-and-Benders-Cut Algorithm with the Master Problem Reformulation Based on the Heuristic Algorithm

This section introduces a strategy to enhance the efficiency of the BBC algorithm by reformulating the master problem. The proposed reformulation integrates information derived from the heuristic algorithm. After this modification, the BBC algorithm proceeds as usual.

The main idea is to increase the duration of all the uncertain tasks by the constant ρ_{heu} , where ρ_{heu} is the stability radius value of the best solution found by the heuristic. This can be achieved by rewriting the master problem as:

$$\text{Maximize } \rho_{heu} + \rho_{inc}$$

We replace τ with $\rho_{heu} + \rho_{inc}$, where ρ_{inc} represents the *increase* in the stability radius from the fixed constant ρ_{heu} , and is treated as a decision variable in the master problem. Consequently, the variable τ is removed.

All the master problem constraints remain unchanged, except for the following capacity constraint:

$$\sum_{j \in V} t_j x_{j,k} \leq T \quad \forall k \in W \quad (2.40)$$

This constraint is now split into the following two constraints:

$$\sum_{j \in V} (t_j + \rho_{heu}) x_{j,k} \leq T \quad \forall k \in \widehat{W} \quad (2.41)$$

$$\sum_{j \in \widetilde{V}^1} (t_j + \rho_{heu}) x_{j,k} + \sum_{j \in V \setminus \widetilde{V}^1} t_j x_{j,k} \leq T \quad \forall k \in W \setminus \widehat{W} \quad (2.42)$$

Constraint (2.41) implies that if a task j is assigned to an uncertain workstation $k \in \widehat{W}$, then its duration must be at least $t_j + \rho_{heu}$. On the other hand, constraint (2.42) captures the following: if an uncertain task $j \in \widetilde{V}^1$ is assigned to a certain workstation $k \in W \setminus \widehat{W}$, then its duration is also considered to be at least $t_j + \rho_{heu}$, whereas if a certain task $j \notin \widetilde{V}^1$ is assigned to a certain workstation, its duration remains unchanged and equal to t_j .

Finally, the Benders optimality cuts will have the following form:

$$\rho_{inc} \leq \sum_{k \in \widehat{W}} \bar{\beta}_k S'_k + \sum_{k \in W \setminus \widehat{W}} \bar{\gamma}_k S''_k + (UB_\infty - \rho_{heu}) \sum_{j \in V, k \in W} \bar{\delta}_{j,k} x_{j,k} \quad (2.43)$$

where

$$S'_k = T - \sum_{j \in V} (t_j + \rho_{heu}) x_{j,k}$$

and

$$S''_k = T - \left[\sum_{j \in \widetilde{V}^1} (t_j + \rho_{heu}) x_{j,k} + \sum_{j \in V \setminus \widetilde{V}^1} t_j x_{j,k} \right]$$

Remark 2.3 Following the same approach as in Remark 2.2, where the cut formulation was simplified, we apply a similar simplification in the current cut (2.43). Let us denote by k^* the workstation responsible for limiting the stability radius. Inequality (2.43) can also be simplified to the following cases. All of that depending on the specific configuration of the sets $\tilde{V}^1$, $\widehat{W}$, and W .

1. **Case 1: The set of uncertain workstations is empty.**

$$\rho_{inc} \leq \frac{1}{\sum_{j \in \tilde{V}^1} \bar{x}_{j,k^*}} \left[S''_{k^*} + (UB_{\infty} - \rho_{heu}) \sum_{\substack{j \in \tilde{V}^1 \\ \bar{x}_{j,k^*}=1}} (1 - x_{j,k^*}) \right] \quad (2.44)$$

2. **Case 2: The set of a priori uncertain tasks is empty.**

$$\rho_{inc} \leq \frac{1}{\sum_{j \in V} \bar{x}_{j,k^*}} \left[S'_{k^*} + (UB_{\infty} - \rho_{heu}) \sum_{\substack{j \in V \\ \bar{x}_{j,k^*}=1}} (1 - x_{j,k^*}) \right] \quad (2.45)$$

3. **Case 3: All workstations are uncertain.** (Exactly as case 2, inequality (2.45))

4. **Case 4: Both certain and uncertain workstations exist, and the set of a priori uncertain tasks are non-empty.**

- (a) If the workstation responsible for limiting the stability radius is uncertain, i.e. $k^* \in \widehat{W}$. (Exactly as case 2, inequality (2.45))
- (b) If the workstation responsible for limiting the stability radius is certain, i.e. $k^* \in W \setminus \widehat{W}$. (Exactly as case 1, inequality (2.44))

2.6.2.1 Computational Results

To evaluate the performance of the Branch-and-Benders-Cut (BBC) algorithm with the new reformulation of the master problem, we conduct experiments on the **SAWYER30** instance, which was introduced earlier. Specifically, we compare between the number of cuts added to the master problem by using BBC algorithm with the new master reformulation, with and without adding heuristic-based cuts to the master problem. Also, we compare the CPU time required to reach the optimal solution in the following four configurations:

- BBC with the new master reformulation, without adding cuts from heuristic solutions to the master problem before starting BBC.
- BBC with the new master reformulation, including cuts generated from heuristic solutions and added to the master problem before starting BBC.
- BBC with the original master formulation, without using the heuristic algorithm (i.e. without adding any cuts to the master problem before starting BBC).
- BBC with the original master formulation, including cuts generated from heuristic solutions and added to the master problem before starting BBC.

Table 2.14 shows that adding heuristic-based cuts to the master problem before applying BBC with the new master reformulation does not lead to a decrease in CPU time compared to the case where these cuts are not added. However, when comparing Table 2.14 with Table 2.15, we observe that applying BBC with the new master reformulation results in a significant reduction in CPU time compared to the original master formulation, especially in instances where the original formulation required considerable computational effort.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Without Heuristic Cuts	15	2	61	20	435	125	124
With Heuristic cuts	22	60	71	10	500	100	119

Table 2.13: Number of cuts added to the master problem by using BBC algorithm with the new master reformulation, with and without adding heuristic-based cuts to the master problem, solved using Gurobi for seven combinations of the **SAWYER30** instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Without Heuristic Cuts	2.2	2.2	8.3	2.1	27.9	14.6	13.3
With Heuristic cuts	2	3.4	8.6	2.4	20.1	9.3	14.7

Table 2.14: CPU time (in seconds) for the BBC algorithm using the new master reformulation, with and without adding heuristic-based cuts to the master problem, solved using Gurobi for seven combinations of the **SAWYER30** instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Without Heuristic Cuts	3.8	1.8	73.7	4.3	45.1	92.8	180.8
With Heuristic Cuts	2.6	2.1	70.1	4.8	56.9	90.9	159

Table 2.15: CPU time (in seconds) for the BBC algorithm using the original master formulation, with and without adding heuristic-based cuts to the master problem, solved using Gurobi for seven combinations of the **SAWYER30** instance.

Algorithm 1: Solving the dual subproblem when $\widehat{W} = \emptyset$

Input: $V, W, \widetilde{V}^1, (t_j)_{j \in V}, (x_{j,k})_{j \in V, W}, T, UB_\infty$
Output: $(\alpha_j)_{j \in V}, (\gamma_k)_{k \in W}, (\delta_{j,k})_{j,k \in V, W}, obj_value$

```

1 foreach  $k \in W$  do  $S[k] = T - \sum_{j \in V} t[j].x[j, k]$ 
2  $results \leftarrow [], gamma\_k\_occurrences \leftarrow \text{defaultdict}(\text{list})$ 
3 foreach  $alpha\_idx \in \widetilde{V}^1$  do
4   foreach  $j \in V$  do  $alpha[j] \leftarrow 0$ 
5   foreach  $k \in W$  do  $gamma[k] \leftarrow 0$ 
6   foreach  $j \in V$  do
7     foreach  $k \in W$  do  $delta[j, k] \leftarrow 0$ 
8   foreach  $j \in V$  do
9     foreach  $k \in W$  do
10       if  $x[j, k] == 1$  then
11          $delta[j, k] \leftarrow 0$ 
12   foreach  $j \in V$  do
13     if  $j \notin \widetilde{V}^1$  then
14        $alpha[j] \leftarrow 0$ 
15       foreach  $k \in W$  do  $delta[j, k] \leftarrow 0$ 
16    $alpha[alpha\_idx] \leftarrow 1$ 
17   foreach  $j \in \widetilde{V}^1$  do
18     if  $j \neq alpha\_idx$  then
19        $alpha[j] \leftarrow 0$ 
20       foreach  $k \in W$  do
21          $delta[j, k] \leftarrow 0$ 
22    $assigned\_k \leftarrow -1$ 
23   foreach  $k \in W$  do
24     if  $x[alpha\_idx, k] \neq 1$  then
25        $delta[alpha\_idx, k] \leftarrow 1$ 
26     else
27        $assigned\_k \leftarrow k$ 
28        $gamma[k] \leftarrow 1$ 
29    $obj\_value \leftarrow gamma[assigned\_k] \cdot S[assigned\_k]$ 
30   Append  $(obj\_value, alpha, gamma, delta)$  to  $results$ 
31   Append  $(alpha\_idx, assigned\_k)$  to  $gamma\_k\_occurrences[assigned\_k]$ 
32 foreach  $(k, occurrences) \in gamma\_k\_occurrences$  do
33   if  $\text{length of } occurrences > 1$  then
34      $num \leftarrow \text{length of } occurrences$ 
35     foreach  $j \in V$  do  $updated\_alpha[j] \leftarrow 0$ 
36     foreach  $k' \in W$  do  $updated\_gamma[k'] \leftarrow 0$ 
37     foreach  $j \in V$  do
38       foreach  $k' \in W$  do  $updated\_delta[j, k'] \leftarrow 0$ 
39     foreach  $(alpha\_idx, \_) \in occurrences$  do
40        $updated\_alpha[alpha\_idx] \leftarrow 1/num$ 
41       foreach  $k' \in W$  do
42         if  $x[alpha\_idx, k'] \neq 1$  then
43            $updated\_delta[alpha\_idx, k'] \leftarrow 1/num$ 
44        $updated\_gamma[k] \leftarrow 1/num$ 
45        $obj\_value \leftarrow updated\_gamma[k].S[k]$ 
46       Append  $(obj\_value, updated\_alpha, updated\_gamma, updated\_delta)$  to  $results$ 
47  $best\_result = \min(results, key = \lambda x : x[0])$ 
48  $best\_obj, best\_alpha, best\_gamma, best\_delta = best\_result$ 
49 return  $best\_alpha, best\_gamma, best\_delta, best\_obj$ 

```

Algorithm 2: Solving the dual subproblem when $\tilde{V}^1 = \emptyset$

Input: $V, W, \hat{W}, (t_j)_{j \in V}, (x_{j,k})_{j \in V, W}, T, UB_\infty$
Output: $(\alpha_j)_{j \in V}, (\beta_k)_{k \in \hat{W}}, (\delta_{j,k})_{j,k \in V, W}, obj_value$

```

1  foreach  $k \in W$  do  $S[k] = T - \sum_{j \in V} t[j].x[j, k]$ 
2  foreach  $j \in V$  do
3    foreach  $k \in W$  do  $\delta[j, k] \leftarrow 0$ 
4   $assigned\_tasks \leftarrow \emptyset$ 
5  foreach  $j \in V$  do
6    foreach  $k \in W$  do
7      if  $x[j, k] == 1$  then
8         $\delta[j, k] \leftarrow 0$ 
9      if  $k \notin \hat{W}$  then
10        $assigned\_tasks \leftarrow assigned\_tasks \cup \{j\}$ 
11  $alpha\_candidates \leftarrow \{j \in V \mid j \notin assigned\_tasks\}$ 
12  $results \leftarrow [], beta\_k\_occurrences \leftarrow \text{defaultdict}(\text{list})$ 
13 foreach  $alpha\_idx \in alpha\_candidates$  do
14   foreach  $j \in V$  do  $alpha[j] \leftarrow 0$ 
15   foreach  $k \in \hat{W}$  do  $beta[k] \leftarrow 0$ 
16   foreach  $j \in V$  do
17     foreach  $k \in W$  do  $current\_delta[j, k] \leftarrow 0$ 
18    $alpha[alpha\_idx] \leftarrow 1$ 
19   foreach  $j \in V$  do
20     if  $alpha[j] == 0$  then
21       foreach  $k \in W$  do  $current\_delta[j, k] \leftarrow 0$ 
22    $assigned\_k \leftarrow -1$ 
23   foreach  $k \in W$  do
24     if  $x[alpha\_idx, k] \neq 1$  then
25        $current\_delta[alpha\_idx, k] \leftarrow 1$ 
26     else
27        $assigned\_k \leftarrow k$ 
28      $beta[k] \leftarrow 1$ 
29    $obj\_value \leftarrow beta[assigned\_k].S[assigned\_k]$ 
30   Append  $(obj\_value, alpha, beta, current\_delta)$  to  $results$ 
31   Append  $alpha\_idx$  to  $beta\_k\_occurrences[assigned\_k]$ 
32 foreach  $(k, alpha\_indices) \in beta\_k\_occurrences$  do
33   if  $\text{length of } alpha\_indices > 1$  then
34      $num \leftarrow \text{length of } alpha\_indices$ 
35     foreach  $j \in V$  do  $updated\_alpha[j] \leftarrow 0$ 
36     foreach  $k' \in \hat{W}$  do  $updated\_beta[k'] \leftarrow 0$ 
37     foreach  $j \in V$  do
38       foreach  $k' \in W$  do  $updated\_delta[j, k'] \leftarrow 0$ 
39     foreach  $j \in alpha\_indices$  do
40        $updated\_alpha[j] \leftarrow 1/num$ 
41       foreach  $k' \in W$  do
42         if  $x[j, k'] \neq 1$  then
43            $updated\_delta[j, k'] \leftarrow 1/num$ 
44      $updated\_beta[k] \leftarrow 1/num$ 
45      $obj\_value \leftarrow updated\_beta[k].S[k]$ 
46     Append  $(obj\_value, updated\_alpha, updated\_beta, updated\_delta)$  to  $results$ 
47  $best\_result = \min(results, \text{key} = \lambda x : x[0])$ 
48  $best\_obj, best\_alpha, best\_beta, best\_delta = best\_result$ 
49 return  $best\_alpha, best\_beta, best\_delta, best\_obj$ 

```

Algorithm 3: Solving the dual subproblem when $\widehat{W} = W$

Input: $V, W, \widetilde{V}^1, \widehat{W}, (t_j)_{j \in V}, (x_{j,k})_{j \in V, W}, T, UB_\infty$
Output: $(\alpha_j)_{j \in V}, (\beta_k)_{k \in W}, (\delta_{j,k})_{j,k \in V, W}, obj_value$

```

1  foreach  $k \in W$  do  $S[k] = T - \sum_{j \in V} t[j].x[j, k]$ 
2   $results \leftarrow []$ 
3   $beta\_k\_occurrences \leftarrow \text{defaultdict}(\text{list})$ 
4  foreach  $alpha\_idx \in V$  do
5      foreach  $j \in V$  do  $alpha[j] \leftarrow 0$ 
6      foreach  $k \in W$  do  $beta[k] \leftarrow 0$ 
7      foreach  $j \in V$  do
8          foreach  $k \in W$  do  $delta[j, k] \leftarrow 0$ 
9      foreach  $j \in V$  do
10         foreach  $k \in W$  do
11             if  $x[j, k] == 1$  then
12                 foreach  $k \in W$  do  $delta[j, k] \leftarrow 0$ 
13          $alpha[alpha\_idx] \leftarrow 1$ 
14         foreach  $j \in V$  do
15             if  $j \neq alpha\_idx$  then
16                 foreach  $k \in W$  do
17                     foreach  $k \in W$  do  $delta[j, k] \leftarrow 0$ 
18          $assigned\_k \leftarrow -1$ 
19         foreach  $k \in W$  do
20             if  $x[alpha\_idx, k] == 1$  then
21                  $assigned\_k \leftarrow k$ 
22                  $beta[k] \leftarrow 1$ 
23             else
24                 foreach  $k \in W$  do  $delta[alpha\_idx, k] \leftarrow 1$ 
25          $obj\_value \leftarrow beta[assigned\_k].S[assigned\_k]$ 
26         Append  $(obj\_value, alpha, beta, delta)$  to  $results$ 
27         Append  $alpha\_idx$  to  $beta\_k\_occurrences[assigned\_k]$ 
28 foreach  $(k, alpha\_indices) \in beta\_k\_occurrences$  do
29     if  $\text{length of } alpha\_indices > 1$  then
30          $num \leftarrow \text{length of } alpha\_indices$ 
31         foreach  $j \in V$  do
32              $updated\_alpha[j] \leftarrow 0$ 
33         foreach  $k' \in W$  do
34              $updated\_beta[k'] \leftarrow 0$ 
35         foreach  $j \in V$  do
36             foreach  $k' \in W$  do
37                  $updated\_delta[j, k'] \leftarrow 0$ 
38         foreach  $j \in alpha\_indices$  do
39              $updated\_alpha[j] \leftarrow 1/num$ 
40             foreach  $k' \in W$  do
41                 if  $x[j, k'] \neq 1$  then
42                      $updated\_delta[j, k'] \leftarrow 1/num$ 
43              $updated\_beta[k] \leftarrow 1/num$ 
44              $obj\_value \leftarrow updated\_beta[k].S[k]$ 
45             Append  $(obj\_value, updated\_alpha, updated\_beta, updated\_delta)$  to  $results$ 
46  $best\_result = \min(results, key = \lambda x : x[0])$ 
47  $best\_obj, best\_alpha, best\_beta, best\_delta = best\_result$ 
48 return  $best\_alpha, best\_beta, best\_delta, best\_obj$ 

```

Algorithm 4: Solving the dual subproblem when $\widehat{W} \neq \emptyset$, $W \neq \emptyset$ and $\widetilde{V}^1 \neq \emptyset$

Input: $V, W, \widetilde{V}^1, \widehat{W}, (t_j)_{j \in V}, (x_{j,k})_{j \in V, W}, T, UB_\infty$
Output: $(\alpha_j)_{j \in V}, (\beta_k)_{k \in \widehat{W}}, (\gamma_k)_{k \in W \setminus \widehat{W}}, (\delta_{j,k})_{j,k \in V, W}, obj_value$

```

1  foreach  $k \in W$  do  $S[k] = T - \sum_{j \in V} t[j] \cdot x[j, k]$ 
2   $results \leftarrow [], dual\_occurrences \leftarrow \text{defaultdict}(\text{list})$ 
3  foreach  $j \in V$  do
4      foreach  $k \in W$  do
5           $initial\_delta[j, k] \leftarrow 1$ 
6          if  $x[j, k] == 1$  then
7               $initial\_delta[j, k] \leftarrow 0$ 
8   $alpha\_forced\_zero \leftarrow \emptyset$ 
9  foreach  $j \in V$  do
10     foreach  $k \in W$  do
11         if  $j \notin \widetilde{V}^1$  and  $k \in W \setminus \widehat{W}$  and  $x[j, k] == 1$  then Add  $j$  to  $alpha\_forced\_zero$ 
12 foreach  $alpha\_idx \in V$  do
13     if  $alpha\_idx \in alpha\_forced\_zero$  then
14         continue
15     foreach  $j \in V$  do  $alpha[j] \leftarrow 0$ 
16     foreach  $k \in \widehat{W}$  do  $beta[k] \leftarrow 0$ 
17     foreach  $k \in W \setminus \widehat{W}$  do  $gamma[k] \leftarrow 0$ 
18      $delta \leftarrow initial\_delta$  (copy)
19      $alpha[alpha\_idx] \leftarrow 1$ 
20     foreach  $j \in V$  do
21         if  $j \neq alpha\_idx$  then
22             foreach  $k \in W$  do  $delta[j, k] \leftarrow 0$ 
23      $assigned\_k \leftarrow -1$ 
24     foreach  $k \in W$  do
25         if  $x[alpha\_idx, k] == 1$  then  $assigned\_k \leftarrow k$ 
26         else  $delta[alpha\_idx, k] \leftarrow 1$ 
27     if  $assigned\_k \in \widehat{W}$  then  $beta[assigned\_k] \leftarrow 1, key \leftarrow ('beta', assigned\_k)$ 
28     else  $gamma[assigned\_k] \leftarrow 1, key \leftarrow ('gamma', assigned\_k)$ 
29      $obj\_value \leftarrow S[assigned\_k]$ 
30     Append  $(obj\_value, alpha, beta, gamma, delta)$  to  $results$ 
31     Append  $alpha\_idx$  to  $dual\_occurrences[key]$ 
32 foreach  $(type, k), alpha\_indices \in dual\_occurrences$  do
33     if length of  $alpha\_indices > 1$  then
34          $num \leftarrow \text{length of } alpha\_indices$ 
35         foreach  $j \in V$  do  $updated\_alpha[j] \leftarrow 0$ 
36         foreach  $k' \in \widehat{W}$  do  $updated\_beta[k'] \leftarrow 0$ 
37         foreach  $k' \in W \setminus \widehat{W}$  do  $updated\_gamma[k'] \leftarrow 0$ 
38         foreach  $j \in V$  do
39             foreach  $k' \in W$  do  $updated\_delta[j, k'] \leftarrow 0$ 
40         foreach  $j \in alpha\_indices$  do
41              $updated\_alpha[j] \leftarrow 1/num$ 
42             foreach  $k' \in W$  do
43                 if  $x[j, k'] \neq 1$  then  $updated\_delta[j, k'] \leftarrow 1/num$ 
44         if  $type == beta$  then  $updated\_beta[k] \leftarrow 1/num$ 
45         else  $updated\_gamma[k] \leftarrow 1/num$ 
46          $obj\_value \leftarrow S[k] \cdot (1/num)$ 
47         Append  $(obj\_value, updated\_alpha, updated\_beta, updated\_gamma, updated\_delta)$  to  $results$ 
48  $best\_result = \min(results, key = \lambda x : x[0])$ 
49  $best\_obj, best\_alpha, best\_beta, best\_gamma, best\_delta = best\_result$ 
50 return  $best\_alpha, best\_beta, best\_gamma, best\_delta, best\_obj$ 

```

Chapter 3

Second Formulation for Maximizing the Robustness of Simple Assembly Lines with Fixed Cycle Time and Limited Number of Workstations

This chapter introduces a new formulation aimed at maximizing the robustness of simple assembly lines with fixed cycle time and a limited number of workstations. The same techniques developed in Chapter 2 are applied here, but in the context of the new model described in the following section.

3.1 Mixed-Integer Linear Programming Formulation for Maximize the Stability Radius

Before introducing the new formulation, we compute u_k , an upper bound on the number of uncertain tasks that can be processed by workstation k . These upper bounds are tighter when a lower bound $LB_\infty \geq 0$ of the stability radius is available (if not, LB_∞ is set to zero).

The lower bound LB_∞ can be obtained using a heuristic algorithm presented previously in Subsection 2.6.1, where the last solution found by the heuristic provides this lower bound LB_∞ .

First, we can observe that $u_k \leq |\tilde{V}_k|$ for all $k \in W$. We order the tasks of $\tilde{V}_k$ by increasing duration, i.e.,

$$t_{\tilde{V}_k[1]} \leq t_{\tilde{V}_k[2]} \leq \dots \leq t_{\tilde{V}_k[|\tilde{V}_k|]},$$

and we define u_k as the largest integer of the set $\{0, \dots, |\tilde{V}_k|\}$ such that:

$$\sum_{i=1}^{u_k} (t_{\tilde{V}_k[i]} + LB_\infty) \leq T.$$

Starting from the original compact formulation of SALBP-S presented previously in Section

2.1, we drop all the ξ variables, and we introduce the z_{ik} variables instead. For all $k \in W$ and $i \in \{1, \dots, |\tilde{V}|\}$, z_{ik} is set to one if and only if there are exactly i uncertain tasks allocated to workstation k .

The new compact formulation of SALBP-S is:

$$\begin{aligned} & \text{Maximize } \rho_\infty \\ & \sum_{k \in Q(j)} x_{jk} = 1, \quad \forall j \in V \end{aligned} \quad (3.1)$$

$$\sum_{q \geq k} x_{iq} \leq \sum_{q \geq k} x_{jq}, \quad \forall (i, j) \in A, \forall k \in Q(i) \cap Q(j) \quad (3.2)$$

$$\sum_{j \in V} t_j x_{j,k} \leq T, \quad \forall k \in W \quad (3.3)$$

$$\sum_{i=1}^{u_k} z_{ik} \leq 1, \quad \forall k \in W \quad (3.4)$$

$$\sum_{j \in \tilde{V}_k} x_{jk} = \sum_{i=1}^{u_k} i \cdot z_{ik}, \quad \forall k \in W \quad (3.5)$$

$$\rho_\infty \leq \frac{1}{i} \left[T - \sum_{j \in V_k} t_j x_{jk} \right] + UB_\infty \left[1 - \sum_{q=i}^{u_k} z_{qk} \right], \quad \forall k \in W, \forall i \in \{1, \dots, u_k\} \quad (3.6)$$

$$x_{jk} \in \{0, 1\}, \quad \forall j \in V, \forall k \in Q(j)$$

$$z_{ik} \in \{0, 1\}, \quad \forall k \in W, \forall i \in \{1, \dots, u_k\}$$

$$\rho_\infty \geq 0$$

Knowing u_k for all $k \in W$, we define binary variables z_{ik} , where z_{ik} is set to one if workstation k processes exactly i uncertain tasks for all $i \in \{1, \dots, u_k\}$. The case where u_k uncertain tasks are allocated to workstation k is modeled by the situation where $z_{ik} = 0$ for all $i \in \{1, \dots, u_k - 1\}$. Constraint (3.4) enforces that at most one variable z_{ik} is set to one for all $k \in W$, while constraint (3.5) ensures that z_{ik} is set to one if and only if workstation k processes exactly i uncertain tasks. It can also be checked that when $z_{ik} = 0$ for all $i \in \{1, \dots, u_k\}$, then workstation k does not process any uncertain task. Finally, constraint (3.6) enforces an upper bound on the stability radius, for each number i of uncertain tasks assigned to workstation k , as the idle time of workstation k divided by i . Note that when z_{ik} are zero, all these inequalities are dominated by $\rho_\infty \leq UB_\infty$, so no upper bound is enforced on the stability radius value when workstation k does not process any uncertain task.

More generally, if $z_{i'k} = 1$, then workstation k processes exactly i' uncertain tasks, and the inequalities built for $i \in \{1, \dots, i' - 1\}$ are dominated by the one obtained for $i = i' - 1$. The inequality obtained for $i = i'$ is for the case where i' uncertain tasks are processed by workstation k , and UB_∞ is added to the right-hand side, which makes the constraint inactive. The same occurs for all the values of i that are larger than or equal to i' . Finally, when $z_{u_k k} = 1$, all the inequalities are dominated by the last one, which corresponds to the case where u_k uncertain tasks are processed by workstation k .

3.2 Benders Decomposition on the Second Formulation to Maximize the Stability Radius

This section focuses on applying Benders Decomposition to the formulation presented in Section 3.1, with the aim of improving computational efficiency. We consider the variables $x_{j,k}$ and $z_{i,k}$ as the complicating variables in the model and we perform a special Benders decomposition in the sense that all the decision variables are in the master problem except ρ_∞ . This is equivalent to relaxing constraint (3.6) and performing a cutting plane approach on the MILP model given in Section 3.1.

We define the master problem to maximize the variable τ , which serves as a provisional objective for ρ_∞ . This variable acts as a placeholder for ρ_∞ during the decomposition process and will be refined through the iterative solution of subproblems and the addition of Benders cuts.

The master problem is formulated as follows:

$$\begin{aligned}
& \text{Maximize } \tau \\
& (3.1) - (3.5) \\
& \tau \leq UB_\infty \\
& x_{jk} \in \{0, 1\}, \quad \forall j \in V, \forall k \in Q(j) \\
& z_{ik} \in \{0, 1\}, \quad \forall k \in W, \forall i \in \{1, \dots, u_k\} \\
& \tau \geq 0
\end{aligned} \tag{3.7}$$

In the above formulation, constraint (3.7) is essential, to avoid unboundedness of our master problem. Moreover, we emphasize that the above formulation includes the complicating variables $x_{j,k}$ and $z_{i,k}$ only. So, it provides a feasible variables $\bar{x}_{j,k}$, $\bar{z}_{i,k}$ and value of $\bar{\tau}$. Then we will use them to solve the subproblem and generate cuts to the master problem if needed.

Our subproblem, which aims to maximize the value of ρ_∞ , can be formulated as follows:

$$\begin{aligned}
& \text{Maximize } \rho_\infty \\
& \rho_\infty \leq a_{ik}, \quad \forall k \in W, \forall i \in \{1, \dots, u_k\} \\
& \rho_\infty \geq 0
\end{aligned} \tag{3.8}$$

where

$$a_{ik} = \frac{1}{i} \left[T - \sum_{j \in V_k} t_j x_{jk} \right] + UB_\infty \left[1 - \sum_{q=i}^{u_k} z_{qk} \right]$$

To derive the dual of our subproblem, we assign the dual variables $\mu_{i,k} \geq 0$ corresponding to each stability constraint, and the dual formulation associated with the primal subproblem becomes:

$$\begin{aligned}
& \text{Minimize } \sum_{k \in W} \sum_{i=1}^{u_k} a_{ik} \mu_{ik} \\
& \sum_{k \in W} \sum_{i=1}^{u_k} \mu_{ik} = 1 \\
& \mu_{ik} \geq 0, \quad \forall k \in W, \forall i \in \{1, \dots, u_k\}
\end{aligned} \tag{3.9}$$

We now proceed with Benders decomposition. Initially, we solve the master problem to obtain a feasible variables $\bar{x}_{j,k}$, $\bar{z}_{i,k}$ and a corresponding value $\bar{\tau}$. We then solve the dual subproblem give these fixed variables $\bar{x}_{j,k}$ and $\bar{z}_{i,k}$. The boundedness of the dual subproblem is guaranteed as the dual variables are constrained to be non-negative, and the quantities

$$T - \sum_{j \in V_k} t_j \bar{x}_{jk}, \quad \forall k \in W$$

and

$$1 - \sum_{q=i}^{u_k} \bar{z}_{qk}, \quad \forall k \in W$$

are strictly positive by construction from the master problem. Therefore, throughout the iterations, only optimality cuts are added to the master problem.

The optimality cuts, which will be added iteratively to the master problem, take the following form:

$$\sum_{k \in W} \sum_{i=1}^{u_k} \left[\frac{\bar{\mu}_{ik}}{i} \left[T - \sum_{j \in V_k} t_j x_{jk} \right] + \bar{\mu}_{ik} \cdot UB_{\infty} \left[1 - \sum_{q=i}^{u_k} z_{qk} \right] \right] \geq \tau \quad (3.10)$$

where $\bar{\mu}_{ik}$ are the dual variables obtained by solving the dual subproblem associated with the current $\bar{x}_{j,k}$ and $\bar{z}_{i,k}$.

The decomposition algorithm proceeds iteratively: the master problem provides a feasible candidates $\bar{x}_{j,k}$, $\bar{z}_{i,k}$ and a candidate value $\bar{\tau}$ for the objective. Based on this solution, we solve the dual subproblem. If the value of the master objective $\bar{\tau}$ equals the optimal objective value of the dual subproblem, then the overall optimal solution has been reached. Otherwise, an optimality cut described in inequality (3.10) is added to the master problem for the next iteration.

Remark 3.1 *As in the case of the original MILP formulation for maximizing the stability radius ρ_{∞} in Section 2.4, we can propose an algorithm to solve the dual subproblem. However, the dual subproblem in the current formulation is significantly simpler. It consists of minimizing the following objective function:*

$$\sum_{k \in W} \sum_{i=1}^{u_k} a_{ik} \mu_{ik}$$

subject to the constraint:

$$\sum_{k \in W} \sum_{i=1}^{u_k} \mu_{ik} = 1.$$

After solving the master problem and obtaining a feasible solution $\bar{x}_{jk}$ and $\bar{z}_{ik}$, we can compute the constants a_{ik} for all $k \in W$ and $i \in \{1, \dots, u_k\}$.

Since the sum of all dual variables μ_{ik} must equal one, the optimal solution to this minimization problem is obtained by assigning the entire weight (i.e., $\mu_{ik} = 1$) to the pair (i, k) corresponding to the smallest value of a_{ik} , and setting all other dual variables to zero. The minimum value of a_{ik} thus directly gives the optimal objective value of the dual subproblem.

3.2.1 Computational Results

The instances JACKSON, MITCHELL, and ROSZIEG, which were previously used to evaluate the stability radius using Benders decomposition in the original MILP formulation (see Subsection 2.3.1), are considered again in this subsection. However, they are now used to evaluate the effectiveness of applying Benders decomposition to the second formulation. For each combination of these instances, we computed the corresponding optimal value of ρ_∞ , the CPU time, and the number of optimality cuts generated and added to the master problem during the Benders decomposition process.

The results, summarized in Tables 3.1, 3.2 and 3.3, show that our decomposition approach yields the same optimal values for the stability radius ρ_∞ as those obtained using the original MILP formulation in Subsection 2.3.1. Furthermore, we observe that the number of optimality cuts required is less compared to the original formulation. Finally, the CPU time is also reduced using the new formulation to maximize the stability radius.

Remark 3.2 *In the previous section, we presented in Remark 3.1 a simple algorithm for solving the dual subproblem. After the computational results, we fixed a feasible assignment obtained from the master problem and used it to solve the dual subproblem, once using a standard solver and once using our proposed algorithm. We observed that the CPU times for both approaches were nearly identical. This is primarily due to the simple structure of our dual subproblem formulation.*

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Optimal ρ_∞	1.5	5.5	1.5	1.25	1.25	0.75	0.75
CPU time (s)	0.07	0.05	0.1	0.14	0.07	0.15	0.18
Number of cuts	3	2	6	8	3	7	8

Table 3.1: Stability of radius ρ_∞ , CPU time (in seconds), number of optimality cuts add to the master problem in order to obtain the optimal solution for each combination of the JACKSON instance using the new formulation.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Optimal ρ_∞	4.25	11.50	3.50	1.75	2.17	1.50	1.13
CPU time (s)	0.52	0.22	0.62	1.22	0.48	1.25	0.96
Number of cuts	11	6	11	17	11	15	13

Table 3.2: Stability of radius ρ_∞ , CPU time (in seconds), number of optimality cuts add to the master problem in order to obtain the optimal solution for each combination of the MITCHELL instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Optimal ρ_∞	3.25	6.50	dnf	1.75	1.75	1.17	0.88
CPU time (s)	1.21	0.18	> 180	1.69	0.56	1.46	3.17
Number of cuts	14	3	–	16	7	17	19

Table 3.3: Stability of radius ρ_∞ , CPU time (in seconds), number of optimality cuts add to the master problem in order to obtain the optimal solution for each combination of the **ROSZIEG** instance.

As we did after applying Benders decomposition to the original formulation, where we applied techniques aimed at improving the efficiency of the Benders decomposition. In the following section, we apply the same techniques, but this time to the decomposition proposed for the second formulation.

3.3 Branch-and-Benders-Cut Algorithm with the Master Problem Reformulation Based on the Heuristic Algorithm on the Second Formulation to Maximize the Stability Radius

Here, we aim to improve the efficiency of Benders decomposition which was applied to the second formulation to maximize the stability radius. As we did in Chapter 2, we now apply the Branch-and-Benders-Cut (BBC) algorithm to enhance the computational performance. This approach avoids re-solving the master problem from scratch each time a new cut is added, significantly accelerating the decomposition process.

Moreover, in the same Chapter 2, we proposed a heuristic algorithm and used it to reformulate the master problem by incorporating information derived from the heuristic. The main idea was to increase the processing times of all uncertain tasks by a constant value ρ_{heu} , where ρ_{heu} is the stability radius of the best solution found by the heuristic. This leads to the following reformulated objective for the master problem:

$$\text{Maximize } \rho_{heu} + \rho_{inc},$$

where ρ_{inc} is a decision variable representing the additional increase in the stability radius beyond the best value obtained by the heuristic (*i.e.* ρ_{heu}). In this reformulation, the original objective variable τ is replaced by $\rho_{heu} + \rho_{inc}$.

In the case of the second formulation, we adopt the same principle. All master problem constraints remain unchanged, except for the capacity constraint of each workstation (Constraint (3.3)), which is split as in the original reformulation into two separate constraints (Constraints (2.41) and (2.42)).

The key change lies in the structure of the Benders optimality cut, which now takes the following form:

$$\sum_{i=1}^{u_k} \left[\sum_{k \in \widehat{W}} \frac{\bar{\mu}_{ik}}{i} S'_k + \sum_{k \in W \setminus \widehat{W}} \frac{\bar{\mu}_{ik}}{i} S''_k \right] + (UB_\infty - \rho_{heu}) \sum_{i=1}^{u_k} \sum_{k \in W} \bar{\mu}_{ik} \left[1 - \sum_{q=i}^{u_k} z_{qk} \right] \geq \rho_{inc} \quad (3.11)$$

where

$$S'_k = T - \sum_{j \in V} (t_j + \rho_{heu}) x_{j,k}$$

and

$$S''_k = T - \left[\sum_{j \in \widetilde{V}^1} (t_j + \rho_{heu}) x_{j,k} + \sum_{j \in V \setminus \widetilde{V}^1} t_j x_{j,k} \right]$$

3.3.1 Computational Results

To evaluate the performance of the Branch-and-Benders-Cut (BBC) approach using the new reformulation of the master problem, we conduct experiments on the **SAWYER30** instance, which was introduced in earlier experiments. In addition, we consider two new instances: **HAHN** and **GUNTHER**.

In the **HAHN** instance, we consider a set of $n = 53$ tasks and $m = 7$ workstations, with a fixed cycle time of $T = 2662.5$ per workstation. In the **GUNTHER** instance, we consider $n = 35$ tasks and $m = 10$ workstations, with a cycle time fixed at $T = 60$. More details about these instances can be found in Appendix B.

Specifically, for each combination in each instance, we compare the CPU time required to reach the optimal value of the stability radius under the following three approaches:

- First Approach: Benders decomposition.
- Second Approach: Branch-and-Benders-Cut algorithm without the new master reformulation.
- Third Approach: Branch-and-Benders-Cut algorithm with the new master reformulation.

Tables 3.4, 3.5, and 3.6 demonstrate that using the Branch-and-Benders-Cut algorithm with the new master reformulation is the most efficient approach for solving the second formulation to maximize the stability radius ρ_∞ .

Combination	$ \widetilde{V}^1 = \lceil \frac{n}{4} \rceil$	$ \widetilde{V}^1 = 0$	$ \widetilde{V}^1 = \lceil \frac{n}{4} \rceil$	$ \widetilde{V}^1 = \lceil \frac{n}{2} \rceil$	$ \widetilde{V}^1 = 0$	$ \widetilde{V}^1 = \lceil \frac{n}{2} \rceil$	$ \widetilde{V}^1 = n$
	$ \widehat{W} = 0$	$ \widehat{W} = \lceil \frac{m}{4} \rceil$	$ \widehat{W} = \lceil \frac{m}{4} \rceil$	$ \widehat{W} = 0$	$ \widehat{W} = \lceil \frac{m}{2} \rceil$	$ \widehat{W} = \lceil \frac{m}{2} \rceil$	$ \widehat{W} = m$
First Approach	18.73	0.95	32.11	36.69	8.21	> 180	> 180
Second Approach	7.1	1.1	9.9	6.7	2.9	14.1	26.4
Third Approach	1.7	0.6	7.2	2.1	6.1	7	20.7

Table 3.4: CPU time (in seconds) for applying Benders decomposition, BBC algorithm without and with the new master formulation on the second formulation to maximize ρ_∞ for seven combinations of **SAWYER30** instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
First Approach	1.66	0.54	3.66	3.09	6.81	8.38	6.51
Second Approach	0.85	0.39	1.29	1.70	1.05	1.30	1.29
Third Approach	0.34	0.49	1.16	0.29	1.10	1.67	0.68

Table 3.5: CPU time (in seconds) for applying Benders decomposition, BBC algorithm without and with the new master formulation on the second formulation to maximize ρ_∞ for seven combinations of HAHN instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
First Approach	4.25	0.49	10.56	100.49	6.80	91.64	> 180
Second Approach	1.49	0.29	4.87	4.85	2.03	13.84	19.41
Third Approach	0.80	0.27	1.65	4.5	2.20	4.83	6.10

Table 3.6: CPU time (in seconds) for applying Benders decomposition, BBC algorithm without and with the new master formulation on the second formulation to maximize ρ_∞ for seven combinations of GUNTHER instance.

Chapter 4

Third Formulation for Maximizing the Robustness of Simple Assembly Lines with Fixed Cycle Time and Limited Number of Workstations

This chapter introduces a third formulation aimed to maximizing the robustness of simple assembly lines with fixed cycle time and a limited number of workstations. The same techniques applied to the first and second formulations in Chapters 2 and 3 are applied here, but in the context of the new model described in the following section.

4.1 Mixed-Integer Linear Programming Formulation for Maximize the Stability Radius

We begin this chapter by introducing the third formulation for maximizing the stability radius. This formulation preserves the core idea introduced in the previous chapter in Section 3.1, of representing the number of uncertain tasks per workstation through discrete modeling, but it encodes these counts using binary expansion variables instead of directly assigning $z_{i,k}$ for each possible count i .

In this third formulation, we start by computing u_k , which is an upper bound on the number of uncertain tasks that can be processed by workstation k . These upper bounds are tighter when a lower bound $LB_\infty \geq 0$ of the stability radius is available (if not, LB_∞ is set to zero).

The lower bound LB_∞ can be obtained using a heuristic algorithm presented previously in Subsection 2.6.1, where the last solution found by the heuristic provides this lower bound LB_∞ . Moreover, the value of u_k can be computed as introduced in Subsection 2.6.1.

Starting from the original compact formulation of SALBP-S presented previously in Section

2.1, we drop all the ξ variables, and instead introduce the z_{bk} variables. For each workstation $k \in W$, the maximum number of uncertain tasks assigned to k is denoted by u_k . If workstation k processes ℓ uncertain tasks, then ℓ is encoded in binary and represented by the sequence of bits $z_{0k}, z_{1k}, \dots, z_{\lfloor \log_2 \ell \rfloor k}$, where z_{0k} corresponds to the least significant bit. Since ℓ cannot exceed u_k , we require $\lfloor \log_2 u_k \rfloor + 1$ binary variables z_{bk} to encode all possible values $\ell \in \{0, \dots, u_k\}$. For an integer ℓ , the set $\zeta_\ell \subseteq \{0, \dots, \lfloor \log_2 u_k \rfloor\}$ denotes the bit positions that must be set to one in the binary representation of ℓ . As a result, this formulation uses $O(n \log_2 n)$ z_{bk} decision variables, which is fewer than the $O(n^2)$ ξ_{jk} variables required in the first formulation.

The third formulation of SALBP-S is:

$$\begin{aligned} \text{Maximize } & \rho_\infty \\ \sum_{k \in Q(j)} x_{jk} &= 1, & \forall j \in V \end{aligned} \quad (4.1)$$

$$\sum_{q \geq k} x_{iq} \leq \sum_{q \geq k} x_{jq}, \quad \forall (i, j) \in A, \forall k \in Q(i) \cap Q(j) \quad (4.2)$$

$$\sum_{j \in V} t_j x_{j,k} \leq T, \quad \forall k \in W \quad (4.3)$$

$$\sum_{j \in \tilde{V}_k} x_{jk} = \sum_{b=0}^{\lfloor \log_2 u_k \rfloor} 2^b \cdot z_{bk}, \quad \forall k \in W \quad (4.4)$$

$$\rho_\infty \leq \frac{1}{\ell} \left[T - \sum_{j \in V_k} t_j x_{jk} \right] + UB_\infty \left[|\zeta_\ell| - \sum_{b \in \zeta_\ell} z_{bk} \right], \quad \forall k \in W, \forall \ell \in \{1, \dots, u_k\} \quad (4.5)$$

$$\begin{aligned} x_{jk} &\in \{0, 1\}, & \forall j \in V, \forall k \in Q(j) \\ z_{bk} &\in \{0, 1\}, & \forall k \in W, \forall b \in \{0, \dots, \lfloor \log_2 u_k \rfloor\} \\ \rho_\infty &\geq 0 \end{aligned}$$

4.2 Benders Decomposition on the Third Formulation to Maximize the Stability Radius

In this section, we proceed in the same way as in the Sections 2.2 and 3.2, where Benders decomposition was applied to the first and second formulations to maximize the stability radius. Specifically, we consider the third formulation for maximizing the stability radius presented in previous Section 4.1, and we apply Benders decomposition to solve it efficiently. We will consider $x_{j,k}$ and $z_{b,k}$ as complicating variables in our model and we perform a special Benders decomposition in the sense that all the decision variables are in the master problem except ρ_∞ . This is equivalent to relaxing constraint (4.5) and performing a cutting plane approach on the MILP model given in Section 4.1.

As in the previous two formulations, we define the master problem to maximize the variable τ , which serves as a provisional objective for the stability radius ρ_∞ . This variable acts as a placeholder for ρ_∞ during the decomposition process and will be refined through the iterative solution of subproblems and the addition of Benders cuts.

The formulation of our master problem will be:

$$\begin{aligned}
& \text{Maximize } \tau \\
& (4.1) - (4.4) \\
& \tau \leq UB_\infty \\
& x_{jk} \in \{0, 1\}, \quad \forall j \in V, \forall k \in Q(j) \\
& z_{bk} \in \{0, 1\}, \quad \forall k \in W, \forall b \in \{0, \dots, \lfloor \log_2 u_k \rfloor\} \\
& \tau \geq 0
\end{aligned} \tag{4.6}$$

In the above formulation, constraint (4.6) is essential, to avoid unboundedness of our master problem. Moreover, we emphasize that the above formulation includes the complicating variables $x_{j,k}$ and $z_{b,k}$ only. So, it provides a feasible variables $\bar{x}_{j,k}$, $\bar{z}_{b,k}$ and value of $\bar{\tau}$. Then we will use them to solve the subproblem and generate cuts to the master problem if needed.

Our subproblem, which aims to maximize the value of ρ_∞ , can be formulated as follows:

$$\begin{aligned}
& \text{Maximize } \rho_\infty \\
& \rho_\infty \leq a_{\ell k}, \quad \forall k \in W, \forall \ell \in \{1, \dots, u_k\} \\
& \rho_\infty \geq 0
\end{aligned} \tag{4.7}$$

where

$$a_{\ell k} = \frac{1}{\ell} \left[C - \sum_{j \in V_k} t_j x_{jk} \right] + UB_\infty \left[|\zeta_\ell| - \sum_{b \in \zeta_\ell} z_{bk} \right], \forall k \in W, \forall \ell \in \{1, \dots, u_k\}$$

Similarly, as in the second formulation, we assign the dual variables $\mu_{\ell,k} \geq 0$ corresponding to each stability constraint to derive the dual of our subproblem. The dual formulation associated with the primal subproblem becomes:

$$\begin{aligned}
& \text{Minimize } \sum_{k \in W} \sum_{\ell=1}^{u_k} a_{\ell k} \mu_{\ell k} \\
& \sum_{k \in W} \sum_{\ell=1}^{u_k} \mu_{\ell k} = 1 \\
& \mu_{\ell k} \geq 0, \quad \forall k \in W, \forall \ell \in \{1, \dots, u_k\}
\end{aligned} \tag{4.8}$$

We now proceed with Benders decomposition. Initially, we solve the master problem to obtain a feasible variables $\bar{x}_{j,k}$, $\bar{z}_{b,k}$ and a corresponding value $\bar{\tau}$. We then solve the dual subproblem give these fixed variables $\bar{x}_{j,k}$ and $\bar{z}_{b,k}$. The boundedness of the dual subproblem is guaranteed as the dual variables are constrained to be non-negative, and the quantities

$$T - \sum_{j \in V_k} t_j \bar{x}_{jk}, \quad \forall k \in W$$

and

$$|\zeta_\ell| - \sum_{b \in \zeta_\ell} \bar{z}_{bk}, \quad \forall k \in W, \forall \ell \in \{1, \dots, u_k\}$$

are strictly positive by construction from the master problem. Therefore, throughout the iterations, only optimality cuts are added to the master problem.

These optimality cuts take the following form:

$$\sum_{k \in W} \sum_{\ell=1}^{u_k} \left[\frac{\bar{\mu}_{ik}}{\ell} \left[T - \sum_{j \in V_k} t_j x_{jk} \right] + \bar{\mu}_{ik} U B_{\infty} \left[|\zeta_{\ell}| - \sum_{b \in \zeta_{\ell}} z_{bk} \right] \right] \geq \tau \quad (4.9)$$

where $\bar{\mu}_{ik}$ are the dual variables obtained by solving the dual subproblem associated with the current $\bar{x}_{j,k}$ and $\bar{z}_{b,k}$.

The decomposition algorithm proceeds iteratively: the master problem provides a feasible candidates $\bar{x}_{j,k}$, $\bar{z}_{b,k}$ and a candidate value $\bar{\tau}$ for the objective. Based on this solution, we solve the dual subproblem. If the value of the master objective $\bar{\tau}$ is equal to the optimal objective value of the dual subproblem, then the overall optimal solution has been reached. Otherwise, an optimality cut described in inequality (4.9) is added to the master problem for the next iteration.

Remark 4.1 *The dual subproblem presented above can be solved using the same simple algorithm outlined in Remark 3.1.*

4.2.1 Computational Results

To assess the effectiveness of applying Benders decomposition to the third formulation, we consider the instances **SAWYER30** and **MITCHELL**, which were previously introduced in earlier experiments. For each combination in each instance, we present the corresponding optimal value of ρ_{∞} , the CPU time, and the number of optimality cuts generated and added to the master problem throughout the Benders decomposition process.

The results, summarized in Tables 4.1 and 4.2, show that our decomposition approach yields the same optimal values for the stability radius ρ_{∞} as those obtained in [3], where the authors used Xpress-Mosel to solve the MILP formulation. However, we observe that for some combinations in the **SAWYER30**, instances, the optimal ρ_{∞} is marked as dnf (did not found), and the corresponding CPU time exceeds 180 seconds. This indicates that our Benders decomposition approach was unable to reach the optimal value of ρ_{∞} within the 3-minute time limit for those combinations.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Optimal ρ_{∞}	4.25	11.5	3.5	1.75	2.17	1.5	1.13
CPU time (s)	0.56	0.25	0.70	1.01	0.47	0.91	0.89
Number of cuts	12	6	15	19	11	16	15

Table 4.1: Stability of radius ρ_{∞} , CPU time (in seconds), number of optimality cuts add to the master problem in order to obtain the optimal solution for each combination of the **MITCHELL** instance using the third formulation.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
Optimal ρ_∞	9.5	23.5	dnf	dnf	6.25	dnf	dnf
CPU time (s)	11.86	1.05	> 180	> 180	3.56	> 180	> 180
Number of cuts	25	11	–	–	17	–	–

Table 4.2: Stability of radius ρ_∞ , CPU time (in seconds), number of optimality cuts add to the master problem in order to obtain the optimal solution for each combination of the **SAWYER30** instance using the third formulation.

As we did in the previous two chapters after applying Benders decomposition to the first and second formulations, where we applied techniques aimed at improving the efficiency of the Benders decomposition. In the following section, we apply the same techniques, but this time to the decomposition proposed for the third formulation.

4.3 Branch-and-Benders-Cut Algorithm with the Master Problem Reformulation Based on the Heuristic Algorithm on the Third Formulation to Maximize the Stability Radius

In order to improve the efficiency of Benders decomposition applied to the third formulation to maximize the stability radius, we will use Branch-and-Benders-Cut (BBC) algorithm to enhance its computational performance, as this approach avoids re-solving the master problem from scratch each time a new cut is added, significantly accelerating the decomposition process.

Moreover, in Chapter 2, we proposed a heuristic algorithm and used it to reformulate the master problem by incorporating information derived from the heuristic. The main idea was to increase the processing times of all uncertain tasks by a constant value ρ_{heu} , where ρ_{heu} denotes the stability radius of the best solution found by the heuristic. We apply the same principle here: instead of maximizing τ , we maximize $\rho_{heu} + \rho_{inc}$, where ρ_{inc} is a decision variable representing the additional increase in the stability radius beyond the best value obtained by the heuristic.

All master problem constraints remain unchanged, except for the capacity constraint of each workstation (Constraint (4.3)), which is split, into two separate constraints (Constraints (2.41) and (2.42)).

The main modification lies in the structure of the Benders optimality cut, which now takes the following form:

$$\sum_{\ell=1}^{u_k} \left[\sum_{k \in \hat{W}} \frac{\bar{\mu}_{ik}}{i} S'_k + \sum_{k \in W \setminus \hat{W}} \frac{\bar{\mu}_{ik}}{i} S''_k \right] + (UB_\infty - \rho_{heu}) \sum_{\ell=1}^{u_k} \sum_{k \in W} \bar{\mu}_{ik} \left[|\zeta_\ell| - \sum_{b \in \zeta_\ell} z_{bk} \right] \geq \rho_{inc} \quad (4.10)$$

where

$$S'_k = T - \sum_{j \in V} (t_j + \rho_{heu}) x_{j,k}$$

and

$$S''_k = T - \left[\sum_{j \in \tilde{V}^1} (t_j + \rho_{heu}) x_{j,k} + \sum_{j \in V \setminus \tilde{V}^1} t_j x_{j,k} \right]$$

4.3.1 Computational Results

To evaluate the performance of the Branch-and-Benders-Cut (BBC) approach with the new reformulation of the master problem applied to the third formulation, we conducted experiments on the **SAWYER30** and **GUNTHER** instances, which were also used in earlier experiments. Specifically, for each combination in each instance, we compare the CPU time required to reach the optimal value of the stability radius under the following three approaches: Benders decomposition (First Approach), BBC without the new master reformulation (Second Approach) and BBC with the new master reformulation (Third Approach).

Tables 4.3 and 4.4 demonstrate that using the Branch-and-Benders-Cut algorithm with the new master reformulation is the most efficient approach for solving the third formulation to maximize the stability radius ρ_∞ .

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
First Approach	11.86	1.05	> 180	> 180	3.56	> 180	> 180
Second Approach	7.18	0.94	7.83	6.85	4.89	10.80	18.64
Third Approach	6.09	0.85	4.73	2.01	6.18	4.75	12.19

Table 4.3: CPU time (in seconds) for applying Benders decomposition, BBC algorithm without and with the new master formulation on the second formulation to maximize ρ_∞ for seven combinations of **SAWYER30** instance.

Combination	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	$ \tilde{V}^1 = n$ $ \hat{W} = m$
First Approach	5.48	0.52	5.78	37.99	2.25	73.05	> 180
Second Approach	1.85	0.31	1.91	4.82	2.73	5.60	14.46
Third Approach	1.15	0.23	1.59	3.59	1.61	2.83	5.49

Table 4.4: CPU time (in seconds) for applying Benders decomposition, BBC algorithm without and with the new master formulation on the second formulation to maximize ρ_∞ for seven combinations of **GUNTHER** instance.

Chapter 5

Comparison Between the Three Formulations for Maximizing the Stability Radius

In this chapter, we compare the three formulations for maximizing the stability radius. For this comparison, **TONGE70**, **BARTHOL2** and **SCHOLL** instances are used. These large instances chosen to highlight the differences between the approaches. In **TONGE70**, there is a set of $n = 70$ tasks and $m = 80$ workstations, with the cycle time fixed at $T = 234$. For **BARTHOL2** instance, there is a set of $n = 148$ tasks and $m = 41$ workstations, with the cycle time fixed at $T = 124.5$. Finally, in **SCHOLL** instance, there is a set of $n = 297$ tasks and $m = 41$ workstations, with the cycle time fixed at $T = 2079$. More details about these instances can be found in [Appendix B](#).

To determine which formulation performs best, we compare the three formulations by reporting, for each, the best ρ_∞ and the corresponding optimality gap obtained within a time limit of 600 seconds, both without decomposition and with decomposition combined with complementary techniques mentioned in the report.

Tables [5.1–5.3](#) show that applying Benders decomposition together with the acceleration techniques presented in this report (master problem reformulation together with the BBC algorithm and our tailored algorithms for solving the dual subproblem) does not deliver a clear efficiency gain for the first, second, or third formulation.

Formulation		First Formulation		Second Formulation		Third Formulation	
Combinations	Comparison	Without Decomposition	With Decomposition	Without Decomposition	With Decomposition	Without Decomposition	With Decomposition
$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	Best ρ_∞	38	37	37	37.5	36.5	36.25
	GAP	0%	61%	52%	49%	89%	101%
	CPU time (s)	556.96	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	Best ρ_∞	106	106	106	106	106	106
	GAP	0%	33%	0%	0%	0%	0%
	CPU time (s)	66.76	dnf	4.84	6.09	5.01	7.46
$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	Best ρ_∞	33	25.25	33	33	33	30
	GAP	0%	183%	15%	50%	21%	53%
	CPU time (s)	66.92	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	Best ρ_∞	19	18	18	18.5	18.5	18
	GAP	6%	166%	127%	151%	132%	177%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	Best ρ_∞	35	28	35	35	35	35
	GAP	0%	173%	0%	0%	0%	0%
	CPU time (s)	228.88	dnf	114.20	377.25	26.01	456.28
$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	Best ρ_∞	15.5	12	14	14.67	15	14
	GAP	12%	323%	128%	263%	158%	299%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = n$ $ \hat{W} = m$	Best ρ_∞	9.67	9	9.2	9.4	9	9
	GAP	4%	206%	97%	182%	129%	230%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf

Table 5.1: The best ρ_∞ and the corresponding optimality gap obtained using the three formulations to maximize the stability radius ρ_∞ without decomposition and with decomposition followed by complementary techniques within a time limit of 600 seconds for seven combinations of TONGE70 instance.

Formulation		First Formulation		Second Formulation		Third Formulation	
Combinations	Comparison	Without Decomposition	With Decomposition	Without Decomposition	With Decomposition	Without Decomposition	With Decomposition
$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	Best ρ_∞	20.5	16.75	20.5	19.5	20.5	19.5
	GAP	15%	148%	97%	113%	111%	113%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	Best ρ_∞	41.5	9.5	60.5	60.5	58.5	60.5
	GAP	89%	733%	28%	31%	32%	31%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	Best ρ_∞	18.75	4.25	18.17	17.5	19.5	16.5
	GAP	25%	132%	125%	137%	116%	152%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	Best ρ_∞	10.25	8.75	9.25	9.25	10.25	9.25
	GAP	15%	284%	256%	263%	232%	263%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	Best ρ_∞	16.75	3.83	16.75	16.75	16.25	14.25
	GAP	108%	981%	133%	147%	158%	191%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	Best ρ_∞	8.5	2.83	9.83	8.5	7.83	8.5
	GAP	38%	1087%	237%	296%	333%	296%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = n$ $ \hat{W} = m$	Best ρ_∞	4.9	4.5	5.13	4.17	4.83	4.83
	GAP	20%	251%	209%	280%	250%	227%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf

Table 5.2: The best ρ_∞ and the corresponding optimality gap obtained using the three formulations to maximize the stability radius ρ_∞ without decomposition and with decomposition followed by complementary techniques within a time limit of 600 seconds for seven combinations of BARTHOL2 instance.

Formulation		First Formulation		Second Formulation		Third Formulation	
Combinations	Comparison	Without Decomposition	With Decomposition	Without Decomposition	With Decomposition	Without Decomposition	With Decomposition
$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = 0$	Best ρ_∞	187.3	161	191	196	193	200.5
	GAP	11%	405%	286%	315%	307%	306%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	Best ρ_∞	549.5	40.63	1032	1032	1111	1032
	GAP	151%	3387%	23%	37%	9%	37%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = \lceil \frac{n}{4} \rceil$ $ \hat{W} = \lceil \frac{m}{4} \rceil$	Best ρ_∞	144	22.13	160	101	162	89
	GAP	44%	3576%	369%	705%	375%	814%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = 0$	Best ρ_∞	82	71.8	71.5	81.25	80	82.5
	GAP	7%	359%	328%	306%	299%	300%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = 0$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	Best ρ_∞	154.2	8.38	175	84	195.5	93.17
	GAP	262%	8761%	295%	783%	267%	697%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = \lceil \frac{n}{2} \rceil$ $ \hat{W} = \lceil \frac{m}{2} \rceil$	Best ρ_∞	70.88	9.25	75	35.25	74.83	56
	GAP	24%	3465%	332%	836%	338%	489%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf
$ \tilde{V}^1 = n$ $ \hat{W} = m$	Best ρ_∞	46.18	41.14	49	46.8	44.7	49.5
	GAP	14%	377%	300%	319%	337%	296%
	CPU time (s)	dnf	dnf	dnf	dnf	dnf	dnf

Table 5.3: The best ρ_∞ and the corresponding optimality gap obtained using the three formulations to maximize the stability radius ρ_∞ without decomposition and with decomposition followed by complementary techniques within a time limit of 600 seconds for seven combinations of SCHOLL instance.

Chapter 6

Conclusion

In this report, we studied three MILP formulations for maximizing the stability radius in assembly lines with a fixed cycle time and a limited number of workstations. Our goal was to improve its computational efficiency. For each formulation, we first applied Benders decomposition to the original model and assessed its correctness by comparing the optimal objective values we obtained with those reported in [3], where the authors used the Xpress Mosel solver to solve the original formulation. Across various instances, the three decompositions returned objective values identical to those reported in [3], confirming the validity of our approaches.

One of the main bottlenecks in Benders decomposition—especially for large instances—is repeatedly solving the dual subproblem at every iteration if the optimal solution hasn’t been reached. To reduce computational effort, we developed four tailored algorithms for solving the dual subproblem of the first formulation and a simple algorithm for the second and third formulations, whose duals are structurally simpler. We embedded these algorithms in place of the LP solver (Gurobi) within the Benders loop. In our experiments, the tailored approaches consistently reduced overall solution time compared with using Gurobi to solve the dual subproblems, especially in the first formulation as the dual subproblem is not as simple as the second and third formulations.

We also observed that, whenever optimality has not yet been reached, the master problem must be resolved from scratch after each newly added cut, which is also computationally expensive. To address this, we adopted a Branch-and-Benders-Cut (BBC) algorithm that integrates Benders decomposition directly into the branch-and-bound tree via lazy-constraint callbacks. Combined with our dual-subproblem algorithms, the BBC approach further decreased the computational effort in our experiments.

We did not stop there; rather, we sought to further improve the efficiency of the Branch-and-Benders-Cut (BBC) algorithm. We use a heuristic algorithm that generates a sequence of improving solutions; for each improvement, we derive and pre-inject the associated optimality cut into the master problem before launching Branch-and-Benders-Cut (BBC) algorithm. The results show that this step alone did not consistently speed up the solution process. Guided by an analysis of these outcomes, we introduced a reformulated master problem that explicitly incorporates information extracted from the proposed heuristic. In our tests, this reformulation used together with Branch-

and-Benders-Cut (BBC) algorithm and our tailored dual algorithms delivered the best overall performance.

To assess which formulation performs best, we test all three by reporting, for each, the best value of ρ_∞ attained within a fixed time limit and the corresponding optimality gap. We evaluated both the original models (without decomposition) and the Benders-decomposed variants enhanced with the complementary acceleration techniques described above. Overall, the results do not show a consistent efficiency advantage for Benders decomposition with accelerations across the three formulations. Therefore, we conclude that for the three formulations, Benders decomposition does not provide a systematic advantage over the non-decomposed formulations.

Appendix A

Algorithm for Solving the Dual Subproblem

In this appendix, we present our first attempt to solve the dual subproblem. We propose an algorithm based on the Projected Gradient Descent (PGD) method. PGD works by iteratively updating the variables in the direction of the negative gradient and then projecting the result back onto the feasible region.

Before introducing the algorithm, it is important to observe that the dual objective function to minimize is convex, and the constraints are linear, this making the dual problem a convex optimization problem. Moreover, based on the structure of the dual constraints (2.29)–(2.32) and our dual objective is to minimize the sum, the variables $\delta_{j,k}$ can be expressed in terms of α_j , β_k , and γ_k as follows:

- For $j \in V$, $k \in \widehat{W}$, we have $-\alpha_j + \beta_k + \delta_{jk} \geq 0$. Therefore:

$$\delta_{j,k} = \max(\alpha_j - \beta_k, 0)$$

- For $k \in W \setminus \widehat{W}$:
 - If $j \in \widetilde{V}^1$, we have $-\alpha_j + \gamma_k + \delta_{jk} \geq 0$. Therefore:

$$\delta_{j,k} = \max(\alpha_j - \gamma_k, 0)$$

- If $j \notin \widetilde{V}^1$, we have $-\alpha_j + \delta_{jk} \geq 0$. Therefore:

$$\delta_{j,k} = \max(\alpha_j, 0)$$

Now we now present the proposed algorithm in a step-by-step manner:

1. Initialization: The algorithm begins by initializing the variables to ensure that they satisfies the dual problem constraints (2.29)–(2.32):
 - (a) $\alpha_j = \frac{1}{|V|}$, for all $j \in V$, providing a uniform distribution that satisfies constraint (2.32) of the dual subproblem.

- (b) $\beta_k = 0$, for all $k \in \widehat{W}$.
- (c) $\gamma_k = 0$, for all $k \in W \setminus \widehat{W}$.
- (d) The initial values of the variables $\delta_{j,k}$ are computed using the structural presented earlier:

$$\delta_{j,k} = \begin{cases} \max(\alpha_j - \beta_k, 0) & \text{if } k \in \widehat{W} \\ \max(\alpha_j - \gamma_k, 0) & \text{if } k \in W \setminus \widehat{W}, j \in \widetilde{V}^1 \\ \max(\alpha_j, 0) & \text{if } k \in W \setminus \widehat{W}, j \notin \widetilde{V}^1 \end{cases}$$

2. Compute objective and gradients: for each iteration:

- (a) Compute $\alpha_j, \beta_k, \gamma_k$ ¹ and $\delta_{j,k}$. Then use them to evaluate the objective:

$$f(\alpha, \beta, \gamma) = \sum_{k \in \widehat{W}} \beta_k \bar{S}_k + \sum_{k \in W \setminus \widehat{W}} \gamma_k \bar{S}_k + \text{UB}_\infty \sum_{j, k \in K, W} \delta_{j,k} \bar{x}_{jk}$$

- (b) Compute the gradients:

- For $j \in V$:

$$\nabla_{\alpha_j} f = \text{UB}_\infty \sum_{k \in W | \bar{x}_{jk}=1} \mathbb{I} \left[\delta_{jk} = \begin{cases} \alpha_j - \beta_k, & \text{if } k \in \widehat{W} \\ \alpha_j - \gamma_k, & \text{if } k \in W \setminus \widehat{W} \text{ and } j \in \widetilde{V}^1 \\ \alpha_j, & \text{if } k \in W \setminus \widehat{W} \text{ and } j \notin \widetilde{V}^1 \end{cases} \right]$$

- For $k \in \widehat{W}$:

$$\nabla_{\beta_k} f = \bar{S}_k - \text{UB}_\infty \sum_{j \in V | \bar{x}_{jk}=1} \mathbb{I}(\delta_{jk} = \alpha_j - \beta_k)$$

- For $k \in W \setminus \widehat{W}$:

$$\nabla_{\gamma_k} f = \bar{S}_k - \text{UB}_\infty \sum_{j \in \widetilde{V}^1 | \bar{x}_{jk}=1} \mathbb{I}(\delta_{jk} = \alpha_j - \gamma_k)$$

3. Updating the dual variables:

- (a) For each $j \in V$, update α_j by:
 - Computing the gradient step:

$$\alpha_j \leftarrow \alpha_j - \eta \cdot \nabla_{\alpha_j} f$$

- Project onto the feasible region: As the updated values of α_j may be violate the constrains ($\sum_j \alpha_j = 1$ and $\alpha_j \geq 0$), we will use the simplex projection algorithm which was presented by Wang and Carreira-Perpiñán in [11] as follows:
 - Sort the updated α_j variables in descending order:

$$\alpha_{(1)} \geq \alpha_{(2)} \geq \dots \geq \alpha_{(n)}.$$

¹We adopt the following notation: the vector of variables α_j is denoted by $\alpha = (\alpha_1, \dots, \alpha_n)$. The same notation applies to the vectors β and γ , i.e. bold symbols such as α, β, γ denote vectors.

- Find the largest $\rho \in \{1, 2, \dots, n\}$ such that:

$$\alpha_{(\rho)} - \frac{1}{\rho} \left(\sum_{i=1}^{\rho} \alpha_{(i)} - 1 \right) > 0.$$

- Compute θ :

$$\theta = \frac{1}{\rho} \left(\sum_{i=1}^{\rho} \alpha_{(i)} - 1 \right).$$

- Project each α_j :

$$\alpha_j \leftarrow \max(\alpha_j - \theta, 0).$$

- (b) For $k \in \widehat{W}$ update β_k and for $k \in W \setminus \widehat{W}$ update γ_k by:

- Computing the gradient step:

$$\beta_k \leftarrow \beta_k - \eta \cdot \nabla_{\beta_k} f, \quad \gamma_k \leftarrow \gamma_k - \eta \cdot \nabla_{\gamma_k} f$$

- Enforce non-negativity:

$$\beta_k = \max(\beta_k, 0), \quad \gamma_k = \max(\gamma_k, 0)$$

- (c) For all $j \in V$ and $k \in W$, update $\delta_{j,k}$ according to the structural rules presented earlier, using the current values of α_j , β_k and γ_k .

In the above procedure, η is the step size and it determines how much the variables are adjusted in the direction of the negative gradient during each iteration. The step size η will be adjusted using a backtracking line search.

The backtracking method begins with a relatively large initial estimate for the step size and progressively reduces it (i.e., “backtracks”) until a sufficient decrease in the objective function is observed. This decrease must be consistent with what is expected based on the step size and the local gradient of the objective function. A common criterion used to determine whether the decrease is sufficient is the Armijo–Goldstein condition [12], [13].

4. Backtracking line search of η with Armijo–Goldstein condition: To ensure the sufficient decrease in the objective function, we must choose the right magnitude of η before accepting the updating value of α_j , β_k and γ_k . So the step size η will be adjusted using a backtracking line search:

- (a) Start with $\eta = 1$.

- (b) Define the two parameters:

- Reduction factor $\tau \in (0, 1)$.
- The parameter $c \in (0, 1)$ which controls the sufficient decrease condition.

- (c) Check the Armijo condition: Verify whether the updated variables with $\eta = 1$, satisfy the following inequality:

$$f(\alpha^k, \beta^k, \gamma^k) \leq f(\alpha^{k-1}, \beta^{k-1}, \gamma^{k-1}) - c\eta \left\| \nabla f(\alpha^{k-1}, \beta^{k-1}, \gamma^{k-1}) \right\|^2$$

- If the condition is satisfied, the current step size η is accepted. Consequently, the updated values of α_j , β_k , γ_k , and $\delta_{j,k}$ corresponding to this η are retained.
- If the condition is not satisfied, the step size η is rejected. The corresponding values of α_j , β_k , γ_k , and $\delta_{j,k}$ are discarded, and η is reduced by multiplying it by the backtracking factor τ :

$$\eta \leftarrow \tau\eta$$

- This process is repeated until the Armijo condition is satisfied. At this point η is accepted and the updated values of α_j , β_k , γ_k , and $\delta_{j,k}$ are retained.
5. Convergence Check: This is the final step. We check whether the absolute difference between the objective value in the current iteration and that in the previous iteration is less than a small predefined tolerance ε . If the following condition is satisfied:

$$\left| f(\boldsymbol{\alpha}^k, \boldsymbol{\beta}^k, \boldsymbol{\gamma}^k) - f(\boldsymbol{\alpha}^{k-1}, \boldsymbol{\beta}^{k-1}, \boldsymbol{\gamma}^{k-1}) \right| < \varepsilon,$$

then the algorithm terminates. Otherwise, the process continues to the next iteration.

It is worth mentioning that, in our case, the backtracking line search with the Armijo–Goldstein condition is guaranteed to terminate and does not result in an infinite loop. This is because the objective function is continuously differentiable, bounded from below, and the chosen search direction namely, the negative of non zero gradient $-\nabla f$ is always a descent direction as:

$$\left\langle \nabla f(\boldsymbol{\alpha}^k, \boldsymbol{\beta}^k, \boldsymbol{\gamma}^k), -\nabla f(\boldsymbol{\alpha}^k, \boldsymbol{\beta}^k, \boldsymbol{\gamma}^k) \right\rangle < 0$$

which ensures that the direction of search leads to a decrease in the objective function [13].

Appendix B

Description of Instances Used in the Experimental Study

In this appendix, we describe all the instances used to conduct the experiments presented in this report. For each instance, we provide the number of tasks n and the number of workstations m , the fixed cycle time T , the vector of nominal task times t and the precedence constraints. Finally, we include the random task permutation and the random workstation permutation used to construct the combinations for each instance.

1. JACKSON Instance

- $n = 11, m = 6, T = 10.5$.
- $t = (6, 2, 5, 7, 1, 2, 3, 6, 5, 5, 4)$.
- Precedence Constraints: $[(1, 2), (1, 3), (1, 4), (1, 5), (2, 6), (3, 7), (4, 7), (5, 7), (6, 8), (7, 9), (8, 10), (9, 11), (10, 11)]$.
- Random Task Permutation: $(6, 4, 9, 8, 5, 1, 2, 7, 3)$.
- Random Workstation Permutation: $(3, 2, 1, 5, 4)$.

2. MANSOOR Instance

- $n = 11, m = 4, T = 67.5$.
- $t = (4, 38, 45, 12, 10, 8, 12, 10, 2, 10, 34)$
- Precedence Constraints: $[(1, 4), (2, 4), (2, 5), (3, 11), (4, 6), (5, 7), (6, 8), (7, 9), (8, 10), (9, 10), (10, 11)]$.
- Random Task Permutation: $(7, 9, 4, 2, 11, 1, 10, 6, 3, 8, 5)$.
- Random Workstation Permutation: $(3, 4, 1, 2)$.

3. MITCHELL Instance

- $n = 21, m = 7, T = 19.5$.
- $t = (4, 3, 9, 5, 9, 4, 8, 7, 5, 1, 3, 1, 5, 3, 5, 3, 13, 5, 2, 3, 7)$.

- Precedence Constraints: [(1, 2), (1, 3), (2, 21), (3, 4), (4, 5), (4, 21), (5, 6), (5, 7), (6, 8), (7, 8), (7, 14), (8, 9), (9, 10), (9, 11), (9, 12), (9, 13), (10, 15), (11, 15), (12, 15), (13, 17), (13, 18), (14, 19), (15, 16), (15, 18), (16, 17), (17, 20), (18, 19)].
 - Random Task Permutation: (14, 10, 16, 5, 8, 7, 17, 2, 3, 15, 19, 18, 12, 21, 4, 11, 13, 9, 1, 20, 6).
 - Random Workstation Permutation: (6, 3, 7, 4, 5, 1, 2).
4. **ROSZIEG** Instance
- $n = 25, m = 8, T = 19.5$.
 - $t = (4, 3, 9, 5, 9, 4, 8, 7, 5, 1, 3, 1, 5, 3, 5, 3, 13, 5, 2, 3, 7, 5, 3, 8, 4)$.
 - Precedence Constraints: [(1, 3), (2, 3), (3, 4), (4, 5), (4, 8), (5, 6), (6, 7), (6, 10), (7, 11), (7, 12), (8, 9), (8, 11), (9, 13), (9, 10), (11, 13), (12, 15), (13, 14), (14, 16), (14, 19), (14, 20), (15, 17), (15, 22), (16, 18), (17, 18), (17, 23), (18, 25), (19, 22), (20, 21), (20, 25), (21, 22), (21, 24), (23, 25)].
 - Random Task Permutation: (14, 23, 19, 15, 11, 8, 17, 6, 24, 5, 1, 4, 7, 25, 16, 2, 13, 21, 18, 12, 20, 22, 9, 10, 3).
 - Random Workstation Permutation: (7, 1, 3, 6, 2, 5, 8, 4).
5. **SAWYER30** Instance
- $n = 30, m = 11, T = 37.5$.
 - $t = (8, 7, 19, 10, 2, 6, 14, 10, 1, 4, 14, 15, 5, 12, 9, 10, 2, 10, 18, 16, 21, 14, 16, 7, 17, 9, 25, 7, 14, 2)$.
 - Precedence Constraints: [(1, 4), (1, 5), (2, 11), (2, 12), (3, 16), (3, 17), (4, 7), (5, 6), (6, 7), (7, 8), (8, 9), (9, 26), (10, 24), (12, 13), (13, 14), (14, 15), (14, 20), (15, 22), (16, 20), (17, 18), (18, 19), (20, 21), (20, 24), (21, 22), (22, 23), (23, 27), (24, 25), (25, 26), (26, 27), (27, 28), (27, 29), (29, 30)].
 - Random Task Permutation: (30, 3, 2, 9, 22, 12, 11, 17, 29, 19, 27, 26, 16, 28, 23, 5, 21, 25, 18, 7, 10, 24, 8, 13, 6, 4, 20, 15, 1, 14).
 - Random Workstation Permutation: (2, 4, 11, 3, 8, 9, 10, 6, 5, 7, 1).
6. **GUNTHER** Instance
- $n = 35, m = 10, T = 60$.
 - $t = (29, 3, 5, 22, 6, 14, 2, 5, 22, 30, 23, 30, 23, 2, 19, 29, 2, 2, 19, 29, 6, 10, 16, 23, 5, 5, 5, 40, 2, 5, 5, 1, 40, 2, 2)$.
 - Precedence Constraints: [(1, 2), (1, 5), (1, 7), (1, 10), (1, 12), (2, 3), (3, 4), (4, 11), (5, 6), (6, 7), (6, 8), (7, 14), (7, 18), (8, 9), (9, 13), (10, 14), (11, 28), (11, 33), (12, 18), (13, 28), (13, 33), (14, 15), (15, 16), (16, 21), (17, 20), (18, 19), (19, 20), (20, 21), (21, 22), (21, 25), (21, 30), (21, 32), (22, 23), (23, 24), (24, 27), (25, 26), (26, 27), (27, 28), (27, 33), (27, 34), (28, 29), (30, 31), (31, 32), (32, 33), (33, 35)].
 - Random Task Permutation: (23, 7, 18, 19, 22, 28, 4, 33, 15, 14, 13, 9, 17, 11, 31, 6, 24, 3, 10, 1, 35, 2, 29, 21, 32, 25, 20, 8, 27, 5, 30, 16, 12, 34, 26)

).

- Random Workstation Permutation: (10, 8, 9, 4, 1, 7, 3, 5, 6, 2).

7. HAHN Instance

- $n = 53, m = 7, T = 2662.5$.
- $t = (971, 142, 142, 142, 103, 96, 99, 1207, 160, 180, 82, 60, 112, 420, 1556, 236, 259, 125, 601, 80, 80, 70, 89, 89, 105, 330, 132, 69, 99, 70, 70, 158, 191, 70, 53, 50, 125, 353, 70, 128, 65, 1775, 91, 91, 113, 487, 138, 80, 80, 65, 40, 742, 1085)$.
- Precedence Constraints: [(1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (2, 36), (3, 36), (4, 9), (5, 9), (6, 9), (7, 9), (8, 9), (9, 10), (10, 11), (11, 12), (12, 13), (12, 14), (12, 15), (13, 16), (13, 17), (13, 18), (14, 22), (15, 16), (15, 17), (16, 29), (16, 30), (16, 31), (17, 19), (18, 42), (19, 20), (19, 21), (19, 22), (20, 23), (21, 24), (22, 25), (23, 36), (24, 36), (25, 26), (26, 27), (27, 28), (28, 29), (28, 30), (28, 31), (28, 32), (28, 33), (28, 34), (29, 35), (30, 35), (31, 35), (32, 35), (33, 35), (34, 35), (35, 36), (36, 37), (37, 38), (37, 39), (37, 40), (38, 41), (39, 41), (40, 41), (41, 42), (42, 43), (42, 44), (42, 45), (42, 46), (42, 47), (43, 48), (44, 49), (45, 51), (45, 52), (46, 50), (47, 51), (47, 52), (48, 51), (48, 52), (49, 51), (49, 52), (50, 51), (50, 52), (51, 53), (52, 53)].
- Random Task Permutation: (5, 41, 50, 53, 20, 22, 21, 6, 29, 9, 35, 11, 13, 8, 49, 10, 38, 34, 32, 25, 46, 36, 23, 30, 19, 18, 7, 26, 24, 37, 51, 16, 39, 43, 28, 12, 17, 52, 14, 15, 2, 1, 44, 45, 4, 47, 40, 3, 33, 48, 42, 31, 27).
- Random Workstation Permutation: (1, 2, 4, 5, 7, 3, 6).

8. TONGE70 Instance

- $n = 70, m = 80, T = 234$.
- $t = (17, 66, 54, 52, 6, 88, 21, 128, 68, 70, 85, 21, 134, 135, 94, 90, 50, 143, 19, 54, 50, 40, 73, 12, 152, 42, 45, 74, 26, 11, 31, 50, 102, 46, 35, 40, 2, 1, 3, 13, 16, 25, 21, 43, 30, 83, 89, 56, 59, 43, 11, 26, 44, 121, 38, 68, 22, 7, 16, 32, 25, 27, 156, 28, 15, 26, 18, 72, 23, 27)$.
- Precedence Constraints: [(1, 2), (1, 41), (1, 69), (1, 70), (2, 3), (3, 4), (3, 68), (4, 6), (4, 7), (5, 6), (5, 24), (5, 30), (6, 8), (7, 8), (8, 12), (9, 10), (10, 11), (11, 12), (12, 13), (12, 14), (13, 23), (14, 23), (15, 16), (16, 17), (16, 18), (17, 19), (18, 19), (19, 20), (19, 22), (19, 57), (20, 21), (21, 23), (22, 23), (23, 25), (23, 31), (23, 33), (24, 25), (25, 26), (25, 27), (25, 28), (25, 29), (26, 35), (27, 35), (28, 35), (29, 35), (30, 31), (31, 32), (32, 35), (33, 34), (34, 35), (35, 36), (35, 44), (35, 48), (35, 51), (35, 53), (35, 56), (35, 60), (35, 61), (35, 62), (36, 37), (37, 38), (38, 39), (39, 40), (40, 42), (41, 42), (42, 43), (43, 50), (44, 45), (45, 46), (46, 47), (47, 50), (48, 49), (49, 50), (51, 52), (52, 54), (53, 54), (54, 55), (57, 58), (58, 59), (59, 60), (61, 65), (62, 63), (63, 64), (64, 65), (64, 66), (64, 67)].
- Random Task Permutation: (49, 16, 13, 57, 11, 9, 63, 24, 59, 14, 21, 69, 36, 55, 37, 7, 15, 39, 33, 46, 52, 67, 5, 3, 19, 45, 28, 25, 4, 48, 22, 31, 56, 61, 10, 23, 12, 66, 34, 54, 50, 38, 62, 65, 30, 40, 43, 47, 68, 51, 58, 42, 44, 70, 20, 26, 27, 17, 41, 8, 64, 32, 1, 60, 18, 29, 2, 53, 35, 6).

- Random Workstation Permutation: (10, 4, 2, 16, 8, 5, 18, 1, 6, 17, 7, 14, 3, 15, 13, 9, 12, 11).

9. BARTHOL2 Instance

- $n = 148$, $m = 41$, $T = 124.5$.
- $t = (16, 30, 7, 47, 29, 8, 39, 37, 32, 29, 17, 11, 32, 15, 53, 53, 8, 24, 24, 8, 7, 8, 14, 13, 10, 25, 11, 25, 11, 29, 25, 10, 14, 41, 42, 47, 7, 80, 7, 41, 47, 16, 32, 66, 80, 7, 41, 13, 47, 33, 34, 11, 18, 25, 7, 28, 12, 52, 14, 3, 3, 8, 16, 33, 8, 18, 10, 14, 28, 11, 18, 25, 40, 40, 1, 5, 28, 8, 81, 7, 26, 10, 21, 26, 20, 21, 47, 23, 13, 19, 15, 35, 26, 46, 20, 31, 19, 34, 51, 39, 30, 26, 13, 45, 58, 28, 8, 83, 40, 34, 23, 62, 11, 19, 14, 31, 32, 26, 55, 31, 32, 26, 19, 14, 19, 48, 55, 8, 11, 27, 18, 36, 23, 20, 46, 64, 22, 15, 34, 22, 51, 48, 64, 70, 37, 64, 78, 78)$.
- Precedence Constraints: [(1, 5), (1, 6), (1, 7), (1, 8), (2, 3), (3, 4), (3, 5), (3, 6), (3, 7), (4, 8), (5, 14), (6, 9), (7, 14), (8, 10), (9, 14), (10, 14), (11, 12), (12, 13), (14, 15), (14, 16), (15, 17), (16, 17), (17, 18), (17, 19), (18, 20), (19, 20), (20, 21), (20, 22), (20, 23), (20, 24), (21, 25), (21, 26), (21, 27), (21, 28), (22, 25), (22, 26), (22, 27), (22, 28), (23, 25), (23, 26), (23, 27), (23, 28), (24, 25), (24, 26), (24, 27), (24, 28), (25, 29), (26, 29), (27, 29), (28, 29), (29, 31), (31, 36), (32, 34), (33, 35), (34, 36), (35, 36), (36, 37), (37, 38), (37, 45), (38, 39), (39, 40), (40, 41), (40, 48), (40, 54), (42, 43), (43, 44), (45, 46), (46, 47), (47, 48), (47, 49), (47, 54), (50, 51), (51, 53), (51, 69), (52, 53), (54, 55), (54, 72), (54, 76), (54, 89), (54, 90), (55, 133), (56, 73), (57, 82), (58, 86), (58, 88), (59, 75), (59, 89), (61, 62), (62, 63), (63, 67), (64, 65), (64, 71), (64, 72), (65, 66), (65, 99), (66, 67), (67, 68), (68, 95), (68, 98), (69, 79), (70, 71), (72, 134), (73, 86), (73, 88), (73, 89), (73, 90), (73, 96), (74, 75), (75, 90), (75, 97), (76, 77), (77, 78), (78, 82), (79, 85), (79, 80), (79, 143), (79, 146), (80, 81), (81, 82), (82, 83), (83, 84), (84, 106), (86, 87), (90, 111), (91, 105), (92, 135), (95, 101), (96, 104), (98, 101), (99, 100), (100, 101), (101, 102), (101, 103), (102, 127), (103, 127), (105, 119), (106, 107), (107, 108), (108, 109), (109, 110), (111, 112), (112, 113), (113, 114), (113, 116), (113, 120), (113, 123), (113, 128), (114, 115), (115, 125), (116, 117), (117, 118), (118, 126), (120, 121), (121, 122), (122, 126), (123, 124), (124, 125), (128, 129), (129, 130), (130, 131), (130, 137), (132, 135), (133, 135), (134, 135), (135, 136), (138, 139), (139, 140), (141, 142), (142, 143), (142, 146), (142, 147), (142, 148), (144, 145), (145, 147), (145, 148)].
- Random Task Permutation: (39, 77, 70, 121, 74, 75, 136, 145, 96, 33, 60, 16, 58, 12, 40, 107, 115, 98, 8, 120, 131, 18, 129, 37, 126, 111, 106, 76, 6, 36, 108, 7, 38, 92, 48, 141, 122, 97, 34, 143, 66, 44, 41, 54, 123, 127, 104, 4, 114, 57, 128, 21, 137, 24, 67, 42, 2, 130, 135, 68, 47, 73, 19, 146, 109, 35, 113, 134, 147, 83, 100, 85, 87, 95, 28, 55, 22, 81, 63, 139, 26, 9, 13, 88, 103, 64, 31, 56, 59, 25, 3, 51, 14, 72, 29, 71, 1, 89, 65, 125, 82, 53, 52, 45, 148, 50, 78, 43, 11, 142, 46, 5, 15, 117, 93, 23, 20, 138, 91, 119, 132, 90, 110, 61, 84, 112, 140, 32, 17, 133, 49, 79, 124, 116, 102, 27, 99, 101, 118, 30, 10, 62, 94, 80, 105, 86, 69, 144).
- Random Workstation Permutation: (34, 19, 25, 32, 29, 4, 37, 12, 1, 13, 21, 30, 11, 35, 23, 28, 5, 8, 17, 10, 22, 40, 36, 7, 14, 31, 24, 2, 16, 41, 27, 6, 20, 9, 18, 39, 38, 33, 15, 3, 26).

10. SCHOLL Instance

- $n = 297, m = 41, T = 2079$.
- $t = (270, 270, 130, 148, 190, 293, 348, 182, 490, 212, 248, 248, 248, 248, 268, 268, 268, 288, 248, 268, 60, 268, 240, 240, 171, 490, 182, 170, 306, 108, 248, 190, 240, 339, 288, 248, 455, 268, 270, 180, 121, 270, 440, 249, 194, 162, 130, 388, 90, 212, 246, 188, 270, 160, 79, 466, 240, 137, 184, 110, 275, 149, 280, 119, 184, 140, 150, 190, 150, 150, 284, 192, 347, 232, 140, 608, 80, 40, 130, 110, 350, 140, 240, 240, 90, 54, 294, 203, 150, 270, 155, 190, 78, 140, 241, 430, 90, 110, 9, 430, 130, 289, 110, 160, 442, 159, 250, 190, 184, 690, 72, 190, 190, 90, 889, 170, 155, 190, 130, 390, 301, 54, 227, 142, 184, 741, 868, 230, 121, 320, 126, 440, 127, 134, 150, 140, 110, 320, 250, 232, 188, 250, 377, 90, 140, 90, 90, 70, 90, 110, 150, 101, 377, 118, 290, 209, 150, 150, 79, 150, 91, 59, 218, 351, 873, 130, 68, 126, 120, 227, 198, 132, 121, 150, 100, 38, 70, 355, 284, 122, 75, 160, 140, 520, 99, 182, 80, 514, 96, 50, 272, 226, 194, 164, 96, 107, 108, 167, 98, 82, 482, 72, 50, 130, 230, 50, 240, 190, 190, 240, 74, 139, 339, 260, 132, 550, 420, 152, 12, 90, 5, 128, 100, 120, 100, 320, 835, 740, 223, 100, 390, 140, 304, 120, 403, 21, 246, 160, 1019, 34, 120, 68, 910, 302, 778, 101, 1310, 20, 278, 81, 290, 100, 372, 72, 28, 90, 250, 144, 303, 220, 58, 224, 211, 99, 44, 120, 70, 421, 231, 214, 196, 280, 398, 72, 280, 356, 193, 140, 130, 300, 456, 7, 170, 252, 210, 308, 308, 121, 52, 426, 104, 1386, 527, 968, 1047, 538).$
- Precedence Constraints: $[(1, 2), (2, 3), (3, 4), (4, 5), (4, 22), (4, 26), (4, 27), (4, 40), (4, 48), (4, 56), (4, 83), (4, 86), (4, 94), (4, 105), (4, 109), (4, 111), (4, 134), (4, 221), (4, 247), (4, 259), (5, 6), (6, 7), (6, 8), (6, 9), (6, 10), (7, 11), (7, 12), (7, 13), (7, 14), (7, 15), (7, 20), (8, 11), (8, 12), (8, 13), (8, 14), (8, 15), (8, 20), (9, 11), (9, 12), (9, 13), (9, 14), (9, 15), (9, 20), (10, 11), (10, 12), (10, 13), (10, 14), (10, 15), (10, 20), (11, 16), (12, 17), (13, 18), (14, 19), (15, 21), (16, 23), (17, 23), (18, 23), (19, 23), (20, 23), (21, 23), (22, 24), (22, 25), (23, 28), (24, 29), (25, 29), (26, 30), (27, 31), (28, 32), (28, 37), (29, 33), (29, 44), (29, 121), (30, 34), (30, 297), (31, 34), (31, 82), (31, 172), (31, 179), (32, 36), (33, 38), (34, 35), (35, 42), (36, 39), (37, 39), (38, 41), (39, 43), (40, 44), (40, 84), (40, 97), (41, 45), (42, 46), (43, 47), (44, 49), (45, 50), (46, 51), (46, 138), (47, 52), (48, 52), (49, 53), (50, 54), (51, 55), (51, 81), (52, 57), (53, 58), (54, 58), (54, 296), (55, 59), (56, 60), (56, 61), (57, 62), (57, 63), (57, 71), (57, 76), (58, 64), (59, 64), (59, 99), (59, 100), (60, 68), (61, 65), (62, 66), (63, 67), (64, 72), (65, 69), (66, 69), (67, 70), (68, 73), (69, 74), (70, 75), (71, 77), (72, 78), (73, 84), (73, 97), (74, 84), (74, 97), (75, 84), (75, 97), (76, 84), (76, 97), (77, 84), (77, 97), (78, 79), (78, 80), (78, 125), (78, 192), (79, 85), (80, 85), (81, 87), (82, 88), (82, 89), (83, 90), (84, 91), (85, 92), (86, 93), (87, 99), (87, 100), (88, 99), (88, 100), (89, 99), (89, 100), (90, 95), (91, 96), (92, 98), (93, 98), (94, 101), (95, 101), (96, 101), (97, 101), (98, 102), (99, 103), (100, 104), (101, 106), (102, 107), (103, 108), (104, 108), (105, 110), (106, 112), (107, 113), (108, 114), (108, 115), (108, 292), (109, 119), (109, 120), (110, 119), (110, 120), (110, 162), (111, 116), (112, 117), (113, 118), (114, 119), (115, 120), (116, 122), (117, 123), (117, 124), (117, 257), (118, 126), (119, 127), (120, 127), (120, 150), (121, 128), (122, 129), (123, 130), (123, 145), (123, 146), (123, 147), (123, 148), (123, 149), (124, 130), (125, 130), (126, 130), (127, 130), (127, 157), (128, 130), (129, 130), (129, 141), (130, 131), (130, 144), (131, 132), (131, 133),$

(132, 135), (133, 135), (133, 170), (134, 136), (135, 137), (136, 139), (137, 140), (138, 140), (138, 191), (139, 142), (139, 253), (140, 143), (140, 200), (141, 151), (142, 152), (143, 153), (143, 169), (144, 154), (145, 155), (146, 156), (147, 158), (148, 159), (149, 160), (150, 161), (151, 163), (152, 164), (153, 165), (154, 166), (155, 166), (156, 166), (157, 166), (158, 166), (159, 166), (160, 166), (161, 166), (162, 167), (163, 166), (164, 167), (165, 168), (165, 176), (166, 170), (167, 171), (168, 173), (169, 174), (170, 174), (171, 174), (172, 175), (173, 177), (174, 178), (174, 287), (174, 288), (175, 180), (176, 181), (177, 181), (178, 181), (179, 181), (180, 181), (180, 252), (181, 182), (181, 183), (181, 184), (181, 185), (181, 186), (181, 187), (181, 188), (181, 189), (181, 196), (181, 197), (181, 295), (182, 190), (183, 193), (184, 194), (185, 195), (186, 195), (187, 195), (188, 195), (189, 195), (190, 195), (191, 200), (192, 201), (193, 198), (194, 199), (195, 199), (195, 203), (195, 205), (195, 227), (195, 229), (196, 202), (197, 202), (198, 202), (199, 202), (200, 202), (201, 202), (202, 204), (202, 251), (203, 206), (203, 208), (204, 207), (204, 250), (205, 207), (206, 209), (207, 210), (207, 212), (208, 210), (209, 210), (209, 211), (210, 213), (211, 213), (212, 214), (213, 214), (214, 215), (214, 234), (215, 216), (216, 217), (217, 218), (218, 219), (219, 220), (220, 222), (221, 223), (222, 224), (223, 225), (224, 226), (225, 227), (226, 228), (227, 230), (228, 231), (229, 235), (229, 236), (230, 232), (230, 271), (230, 289), (231, 233), (232, 236), (233, 237), (234, 238), (234, 256), (235, 237), (236, 239), (237, 240), (238, 240), (238, 285), (239, 240), (239, 279), (240, 241), (240, 243), (241, 242), (242, 244), (243, 245), (243, 246), (244, 245), (244, 246), (244, 255), (245, 248), (246, 248), (247, 278), (248, 249), (249, 254), (249, 284), (250, 256), (251, 256), (252, 258), (253, 260), (254, 261), (255, 261), (255, 262), (256, 263), (257, 264), (258, 265), (259, 266), (260, 267), (261, 268), (261, 269), (262, 269), (263, 270), (264, 271), (265, 272), (266, 271), (267, 273), (268, 274), (269, 274), (270, 274), (271, 274), (272, 275), (273, 276), (274, 277), (274, 278), (274, 282), (275, 280), (276, 281), (277, 283), (278, 283), (279, 286), (280, 290), (281, 291), (282, 293), (283, 293), (284, 294), (285, 294), (286, 294), (287, 293), (288, 293), (289, 294), (290, 293), (291, 293), (292, 293)].

- Random Task Permutation: (121, 53, 239, 163, 222, 272, 207, 175, 15, 271, 285, 250, 74, 168, 228, 214, 263, 196, 256, 30, 211, 147, 144, 227, 201, 106, 46, 141, 52, 184, 177, 146, 219, 240, 205, 249, 125, 283, 171, 216, 68, 223, 234, 10, 29, 145, 274, 48, 209, 19, 108, 194, 111, 182, 183, 160, 77, 269, 200, 112, 85, 41, 237, 2, 25, 257, 124, 235, 164, 149, 270, 24, 136, 9, 169, 203, 291, 210, 220, 76, 57, 23, 99, 88, 42, 225, 64, 120, 162, 16, 122, 155, 62, 34, 289, 116, 148, 60, 245, 102, 248, 32, 114, 246, 231, 142, 199, 193, 192, 258, 118, 190, 189, 1, 130, 84, 265, 153, 204, 259, 101, 104, 218, 134, 113, 82, 47, 20, 81, 282, 224, 58, 33, 296, 91, 229, 131, 281, 186, 27, 179, 253, 65, 35, 247, 51, 254, 4, 129, 242, 73, 208, 161, 295, 137, 212, 241, 105, 266, 45, 151, 275, 103, 8, 31, 264, 152, 233, 119, 260, 244, 92, 202, 133, 37, 284, 59, 66, 273, 174, 195, 159, 49, 70, 187, 252, 166, 165, 191, 226, 79, 36, 67, 185, 294, 83, 7, 55, 251, 293, 297, 39, 78, 96, 97, 69, 197, 279, 157, 206, 90, 178, 213, 110, 287, 56, 180, 93, 11, 107, 44, 126, 286, 109, 80, 17, 139, 150, 292, 167, 172, 230, 94, 143, 238, 5, 40, 232, 135, 18, 138, 13, 75, 236, 127, 117, 156, 217, 3, 95, 288, 12, 278, 262, 215, 22, 132, 86, 173, 38, 261, 176, 71, 140, 170, 181, 54, 123, 21, 61, 6, 277, 87, 115, 154, 290, 26, 280, 89, 243,

98, 128, 72, 188, 28, 255, 63, 198, 158, 268, 221, 43, 100, 14, 276, 50, 267).

- Random Workstation Permutation: (27, 14, 28, 4, 35, 34, 11, 30, 9, 20, 13, 29, 1, 36, 32, 22, 17, 41, 12, 10, 21, 2, 15, 33, 40, 18, 24, 5, 8, 19, 37, 6, 38, 31, 39, 26, 16, 23, 3, 25, 7).

Bibliography

- [1] Nikola Tomac, Radoslav Radonja, and Jasminka Bonato. “Analysis of Henry Ford’s contribution to production and management”. In: *Pomorstvo* 33 (June 2019), pp. 33–45. DOI: [10.31217/p.33.1.4](https://doi.org/10.31217/p.33.1.4).
- [2] Armin Scholl and Christian Becker. “State-of-the-art exact and heuristic solution procedures for simple assembly line balancing”. In: *European Journal of Operational Research* 168.3 (2006). Balancing Assembly and Transfer lines, pp. 666–693. ISSN: 0377-2217. DOI: <https://doi.org/10.1016/j.ejor.2004.07.022>. URL: <https://www.sciencedirect.com/science/article/pii/S0377221704004795>.
- [3] André Rossi et al. “Maximizing the robustness for simple assembly lines with fixed cycle time and limited number of workstations”. In: *Discrete Applied Mathematics* 208 (2016), pp. 123–136. ISSN: 0166-218X. DOI: <https://doi.org/10.1016/j.dam.2016.03.005>. URL: <https://www.sciencedirect.com/science/article/pii/S0166218X16301251>.
- [4] İlker Baybars. “A Survey of Exact Algorithms for the Simple Assembly Line Balancing Problem”. In: *Management Science* 32.8 (1986), pp. 909–932. ISSN: 00251909, 15265501. URL: <http://www.jstor.org/stable/2631657> (visited on 07/18/2025).
- [5] Yuri N. Sotskov, Alexandre Dolgui, and Marie-Claude Portmann. “Stability analysis of an optimal balance for an assembly line with fixed cycle time”. In: *European Journal of Operational Research* 168.3 (2006). Balancing Assembly and Transfer lines, pp. 783–797. ISSN: 0377-2217. DOI: <https://doi.org/10.1016/j.ejor.2004.07.028>. URL: <https://www.sciencedirect.com/science/article/pii/S0377221704004850>.
- [6] Alexandre Dolgui, Jerzy Soldek, and Oleg Zaikin. *Supply Chain Optimisation*. Boston, MA: Springer US, 2005. DOI: [10.1007/b101812](https://doi.org/10.1007/b101812).
- [7] Evgeny Gurevsky, Olga Battaïa, and Alexandre Dolgui. “Balancing of simple assembly lines under variations of task processing times”. In: *Annals of Operations Research* 201 (Dec. 2012). DOI: [10.1007/s10479-012-1203-5](https://doi.org/10.1007/s10479-012-1203-5).
- [8] Evgeny Gurevsky, Olga Battaïa, and Alexandre Dolgui. “Stability measure for a generalized assembly line balancing problem”. In: *Discrete Applied Mathematics* 161.3 (2013), pp. 377–394. ISSN: 0166-218X. DOI: <https://doi.org/10.1016/j.dam.2012.08.037>. URL: <https://www.sciencedirect.com/science/article/pii/S0166218X1200337X>.

- [9] Zeki Caner Taşkin. “Benders Decomposition”. In: *Wiley Encyclopedia of Operations Research and Management Science*. John Wiley Sons, Ltd, 2011. ISBN: 9780470400531. DOI: <https://doi.org/10.1002/9780470400531.eorms0104>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9780470400531.eorms0104>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9780470400531.eorms0104>.
- [10] J. F. Benders. “Partitioning procedures for solving mixed-variables programming problems”. In: *Numerische Mathematik* 4.1 (1962), pp. 238–252. DOI: [10.1007/bf01386316](https://doi.org/10.1007/bf01386316).
- [11] Weiran Wang and Miguel Á. Carreira-Perpiñán. *Projection onto the probability simplex: An efficient algorithm with a simple proof, and an application*. 2013. arXiv: [1309.1541](https://arxiv.org/abs/1309.1541) [cs.LG]. URL: <https://arxiv.org/abs/1309.1541>.
- [12] Larry Armijo. “Minimization of functions having Lipschitz continuous first partial derivatives.” In: *Pacific Journal of Mathematics* 16.1 (1966), pp. 1–3.
- [13] Nicholas Gould and Sven Leyffer. “An Introduction to Algorithms for Nonlinear Optimization”. In: *Frontiers in Numerical Analysis* (Jan. 2003). DOI: [10.1007/978-3-642-55692-0_4](https://doi.org/10.1007/978-3-642-55692-0_4).